

Sistemas Distribuidos I [TA050]

TP Tolerancia a Fallos Coffee Shop Analysis

Grupo Kafcafé

Integrante	Padrón	Email
Castro Martinez, Jose Ignacio	106957	jcastrom@fi.uba.ar
Diem, Walter Gabriel	105618	wdiem@fi.uba.ar
Gestoso, Ramiro	105950	rgestoso@fi.uba.ar

Índice

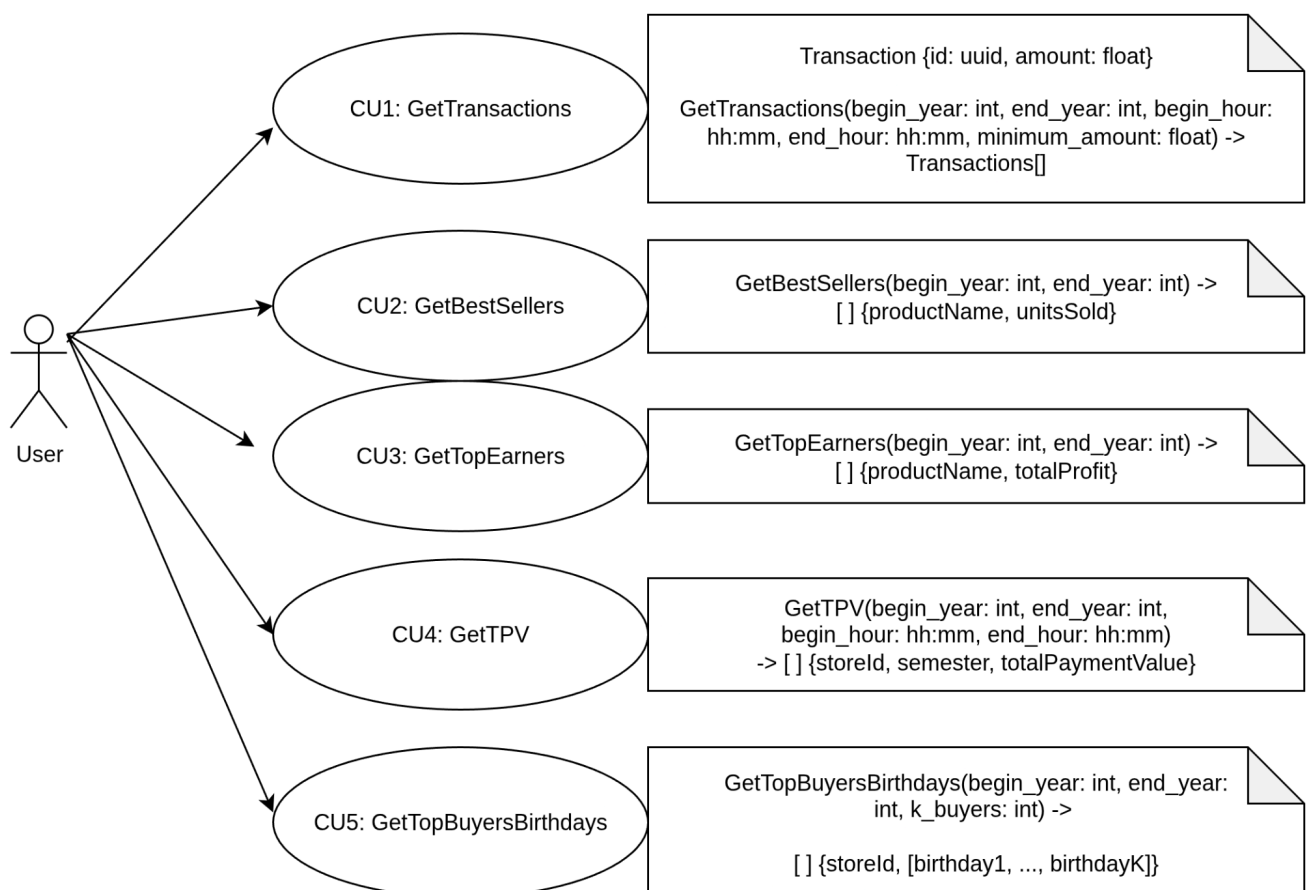
1. Alcance	3
Diagrama de casos de uso	3
2. Objetivos arquitecturales y limitaciones	5
Objetivos	5
Limitaciones	5
3. Vista lógica	7
DAG (Directed Acyclic Graph)	7
4. Vista de procesos	9
Diagramas de actividad	9
Queries	9
Esquema de manejo de EOF	14
Independencia entre capas	21
Envío de datos del cliente	21
WatchMesh: resurrección, elecciones y heartbeat	24
Diagramas de secuencia	30
Queries	30
5. Vista de desarrollo	35
Diagrama de paquetes	35
6. Vista Física	39
Diagrama de robustez	39
Query 1	39
Query 2	40
Query 3	41
Query 4	42
Proceso de consenso para batch terminado	42
Diagrama de despliegue	43
7. División de tareas	48
8. Apéndice: Tolerancia a fallos	50
Secciones de fallos	50
Persistencia en el sistema	50
Nodos Filtros	51
Nodos Group	51
Nodos Group Top K	51
Nodos Join	51
Idempotencia	52
Cache del sistema	57
Crasher	59
Simulación de caos	61
9. Referencias	62

1. Alcance

El trabajo comprende los siguientes casos de uso:

1. Obtener transacciones (id y monto) realizadas durante 2024 y 2025 entre las 06:00 AM y las 11:00 PM con monto total mayor o igual a 75.
2. Obtener productos más vendidos (nombre y cantidad) y productos que más ganancias han generado (nombre y monto), para cada mes en 2024 y 2025.
3. Obtener TPV (Total Payment Value) por cada semestre en 2024 y 2025, para cada sucursal, para transacciones realizadas entre las 06:00 AM y las 11:00 PM.
4. Obtener fecha de cumpleaños de los 3 clientes que han hecho más compras durante 2024 y 2025, para cada sucursal.

Diagrama de casos de uso



Se agregó como notas a los casos de uso una notación en pseudocódigo de cómo se vería el caso de uso como una función, y cuál sería el valor de retorno de la misma.

2. Objetivos arquitecturales y limitaciones

Objetivos

Escalabilidad Horizontal Multicomputing

El sistema debe poder incorporar nuevos nodos de procesamiento en un entorno multicomputing para manejar mayores volúmenes de datos.

Procesamiento Distribuido de la Información

El sistema debe permitir procesar grandes volúmenes de datos transaccionales (ventas, clientes, productos, sucursales) de manera paralela y eficiente.

Abstracción de Comunicación entre Componente

Todo el intercambio de información entre nodos debe realizarse mediante un *MOM* (Message Oriented Middleware, el cual será RabbitMQ), asegurando desacoplamiento y transparencia de distribución.

Transparencia de Acceso y Localización

Los usuarios (consultas al sistema) no deben percibir dónde ni cómo se ejecuta el análisis: el sistema actúa como una unidad lógica homogénea.

Compatibilidad con formatos de datasets estándar

El sistema debe poder trabajar directamente con los archivos provistos (transacciones, items, usuarios, tiendas, productos) del dataset especificado en Kaggle en formato CSV.

Multi-Cliente

El sistema debe poder atender a más de un cliente de forma simultánea y recibir sus resultados independientemente de los datos provistos por otros clientes.

Tolerancia a fallos

El sistema debe ser tolerante a fallos por caída de procesos y poder continuar el procesamiento sin que el usuario lo note.

Limitaciones

Fuente de Datos Restringida

El sistema se diseña exclusivamente sobre el formato CSV del dataset proporcionado por la cátedra.

Ejecución única de procesamiento

El diseño actual comprende un único cliente que ejecuta un único procesamiento de los datos, contemplando este todas las queries en simultáneo.

Tipos de consultas acotados al alcance

El sistema puede resolver las consultas especificadas en los casos de uso, con una determinada flexibilidad para cambiar los parámetros de consulta. No se contempla un uso genérico del sistema como lo sería un DBMS SQL.

Multi-Cliente

El sistema puede atender una cantidad limitada de clientes configurable. Si el límite se excede, los nuevos clientes quedarán en espera a que el servidor los autorice.

Tolerancia a fallos

El sistema puede resolver caídas en los nodos de procesamiento hasta que quede una instancia. Si se caen todas, no existe una forma de revivirlas de manera automática.

El sistema NO admite caídas de: cliente¹, servidor² (client handler) o RabbitMQ.

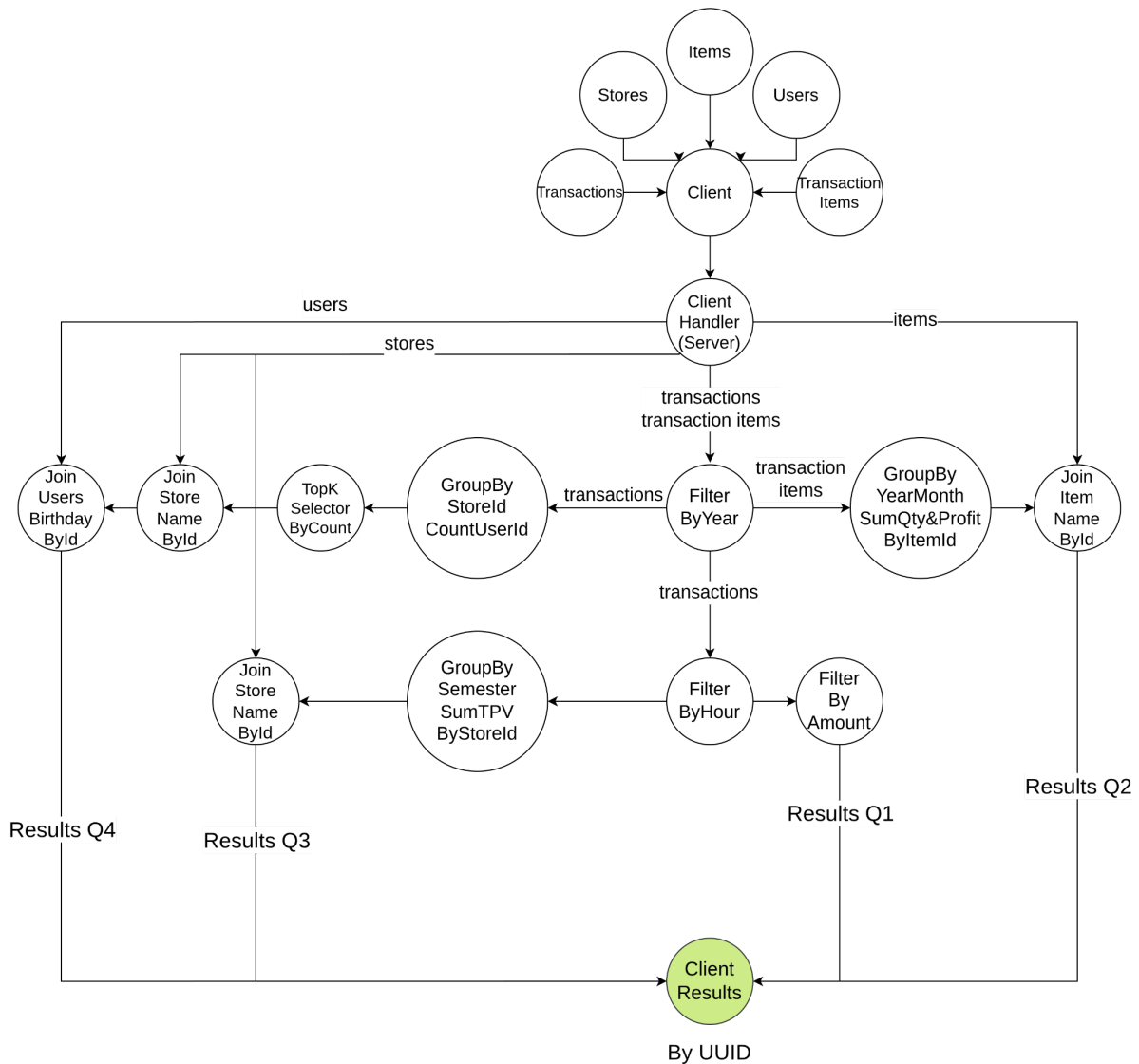
¹ Fue considerado pero no se llegó a tiempo a desarrollarlo por completo.

² Fue considerado pero no implementado. Existen ideas para realizarlo.

3. Vista lógica

DAG (Directed Acyclic Graph)

Se presenta el grafo dirigido acíclico que expone el flujo lógico de la aplicación a gran escala:



Inicia el cliente con todos los archivos necesarios para resolver las queries: stores, items, users, transactions y transaction items. Todos estos son enviados al servidor para su procesamiento.

Luego, el servidor envía lo que son las tablas auxiliares (stores, items, users) a los nodos de tipo join. También envía transactions y transactions items al primer nodo de procesamiento que es el filtro por año.

Cada nodo luego deposita sus resultados y son leídos por quien los requiera. Con esta estructura se busca demorar la duplicación de información lo más posible, para reducir volumen de procesamiento, congestión de red y latencia.

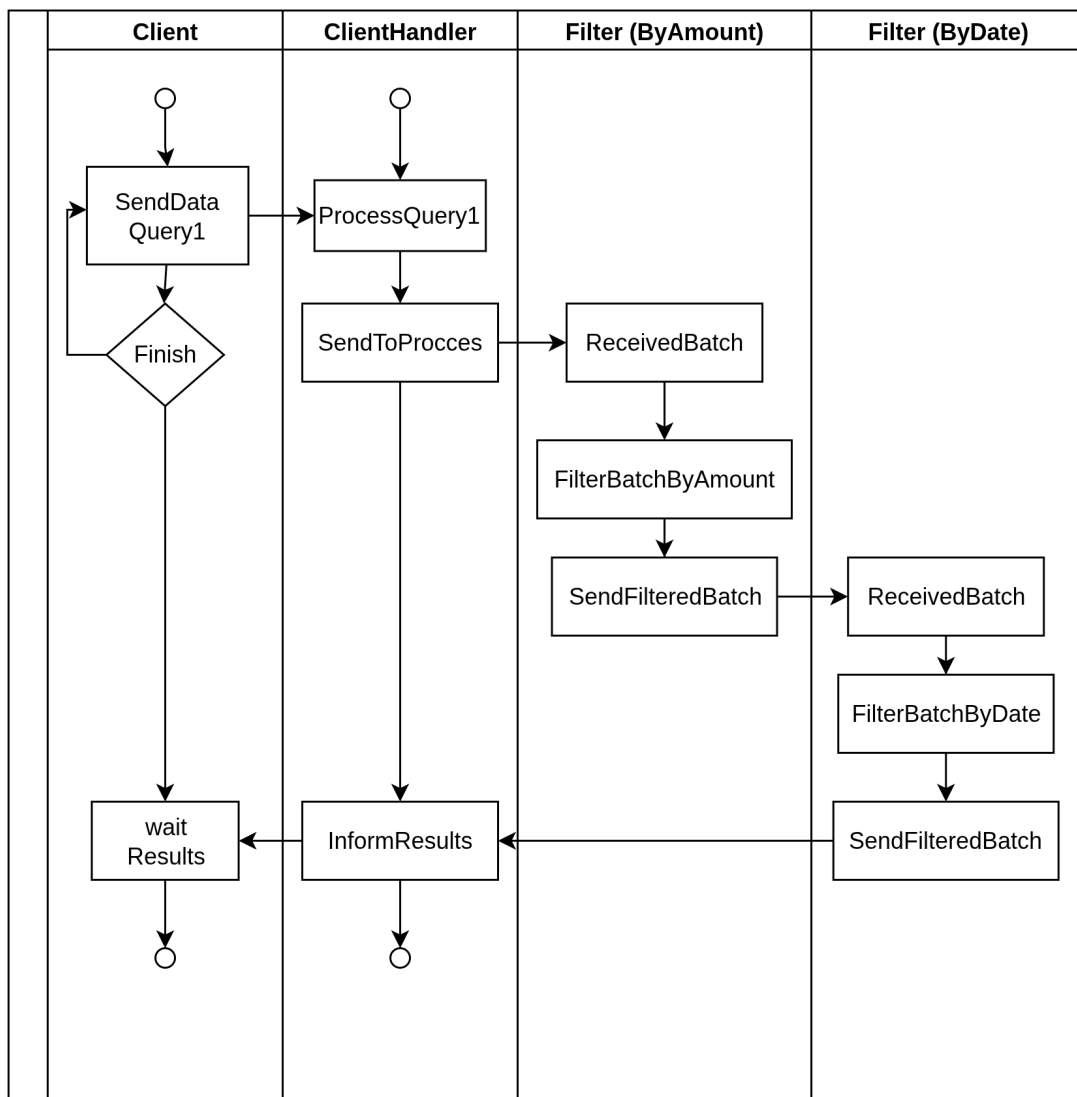
Al final, algunos nodos joins serán los últimos de la cadena de procesamiento y así responsables de depositar los resultados en una cola reservada para el cliente, leída por el servidor.

4. Vista de procesos

Diagramas de actividad

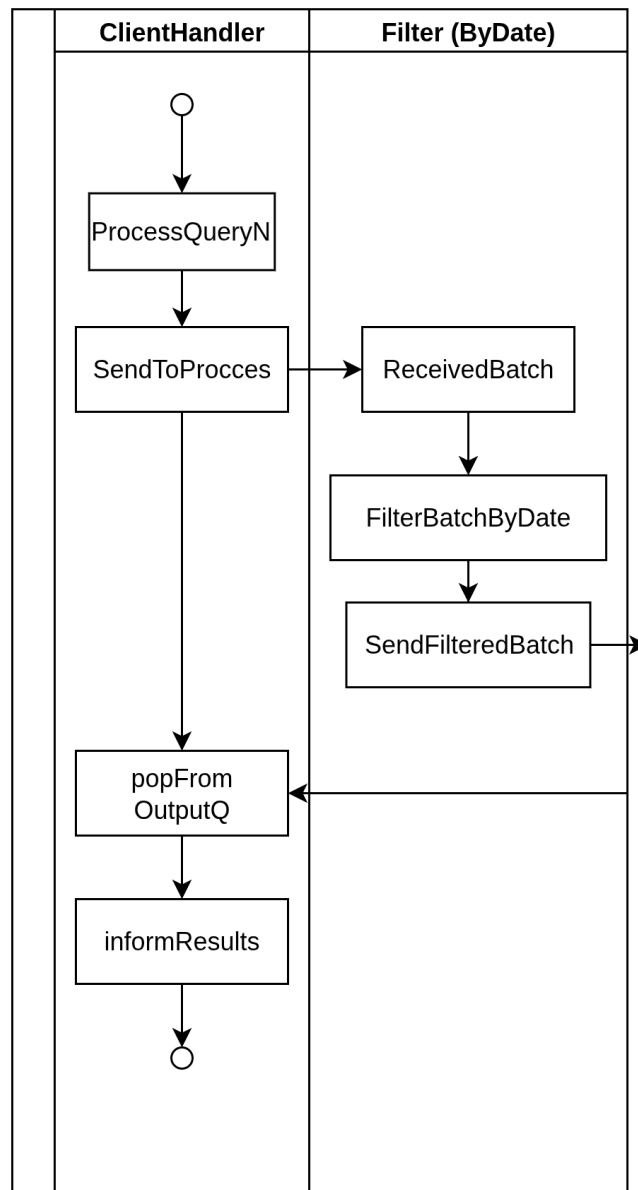
Queries

Se presenta a continuación la representación del procesamiento íntegro de la query número 1 y el flujo que tiene la información desde el cliente hasta obtener el resultado.

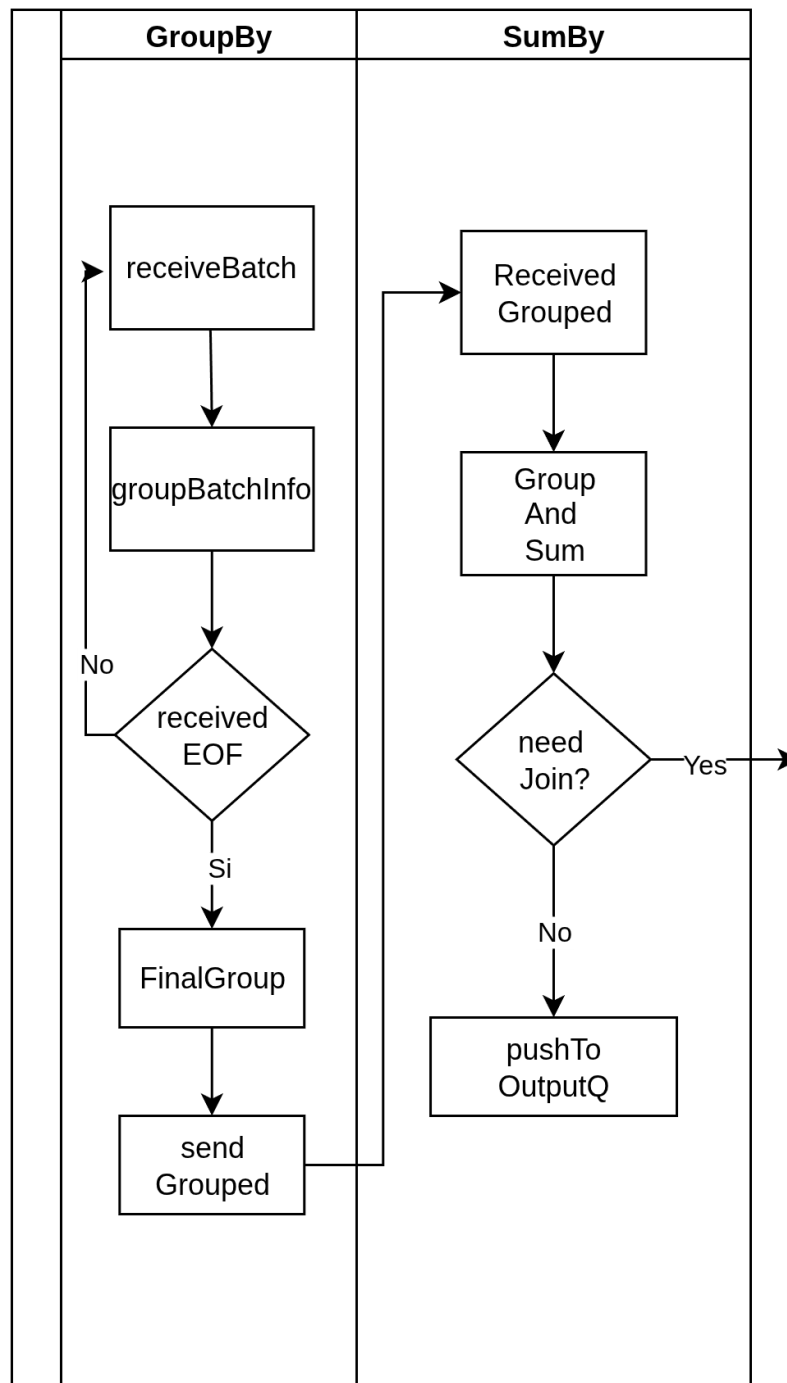


Por otro lado, se agrupa el comportamiento inicial para cada una de las consultas en donde, el Nodo **ClientHandler** se comunica inicialmente con un Nodo **Filter**, diferenciándose solamente en la Query número 1, donde el primer filtro no es una fecha

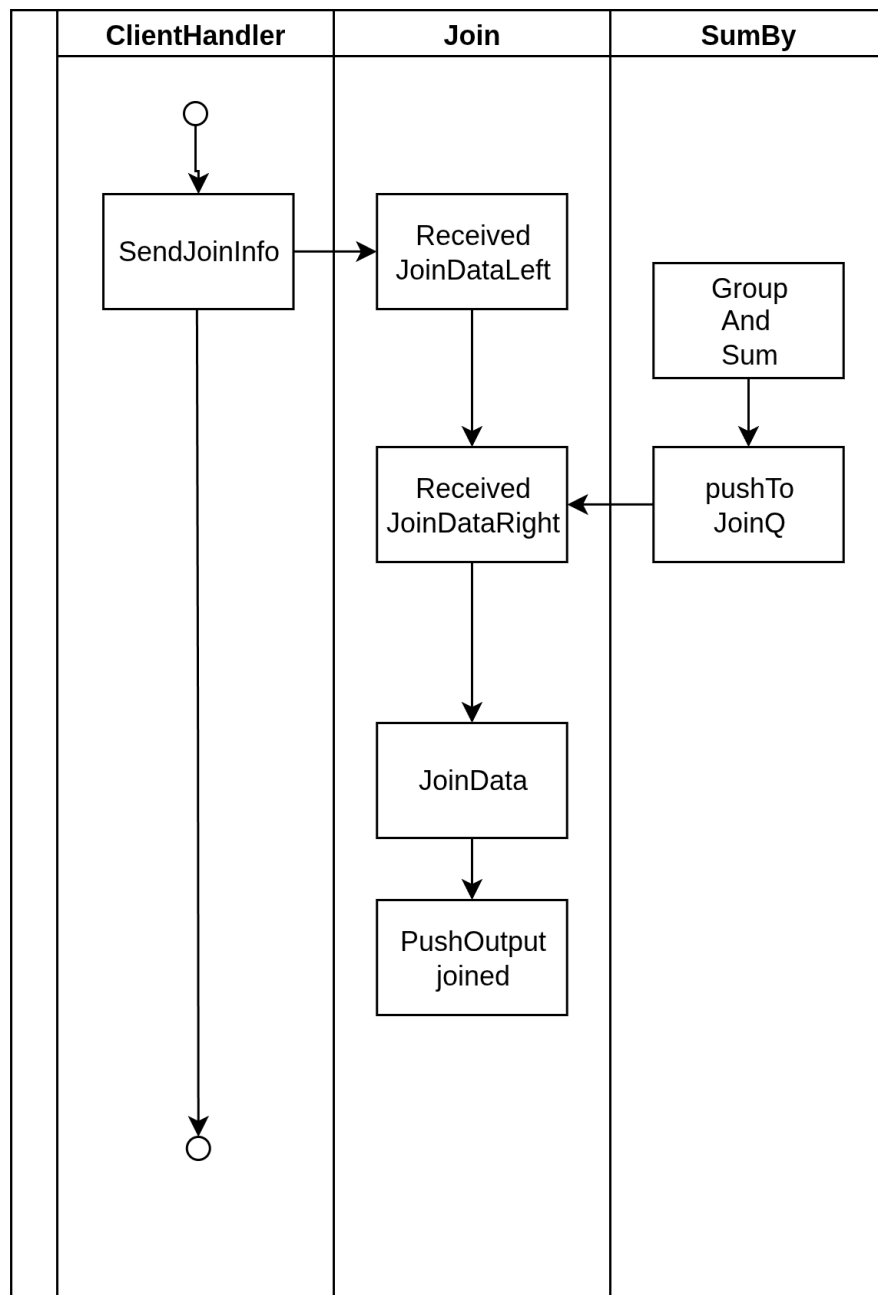
si no una cantidad. Una vez pasada la información por la capa de los filtros se continúa con el procesamiento en los siguientes nodos.



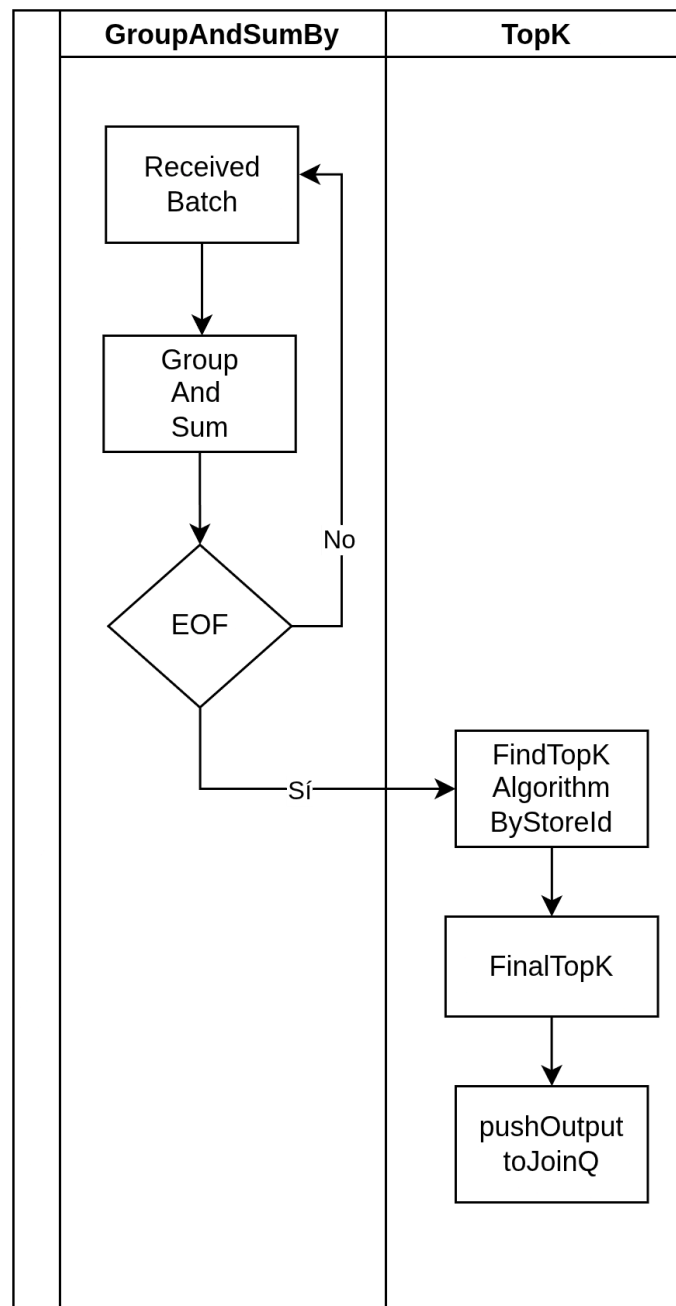
Correspondientes con todas las consultas con excepción de la primera, después de una primera etapa de filtrado se procede a realizar una etapa de agrupamiento y reducción de dimensionalidad de los datos, véase como un ejemplo la consulta número 2, donde se agrupa por mes y año en un primer paso y posteriormente se agrupa la información según el Id del item, sumando en cada caso la cantidad y las ganancias de cada registro. En el siguiente diagrama se observa este comportamiento de forma genérica para las queries que requieran agrupamiento, se hace una separación de las dos fases de forma lógica:



Por otro lado, el comportamiento de las operaciones de junta resulta homogéneo, necesitando en un principio la información cruda proveniente desde el cliente la cual recibe desde el principio del inicio de la transacción para posteriormente esperar el resultado del procesamiento de otros nodos. Una vez el nodo realice la junta entre los dos conjuntos de datos que posee procede a pushear a la cola de resultados para ser despachados al cliente.



Finalmente, se grafica el comportamiento esperado para realizar las operaciones Top, este comportamiento solo se da en la consulta número 4 y el nodo que realice esta operación siempre tendrá provista su información por la capa lógica previa GroupAndSumBy. Perteneciendo toda esta funcionalidad en un mismo nodo para reducir altos costes de envío de datos poco reducidos en volumen, dado que para hacer una operación TopK con un resultado óptimo se requiere tener la totalidad de datos.



Esquema de manejo de EOF

Entidades presentes en los próximos diagramas:

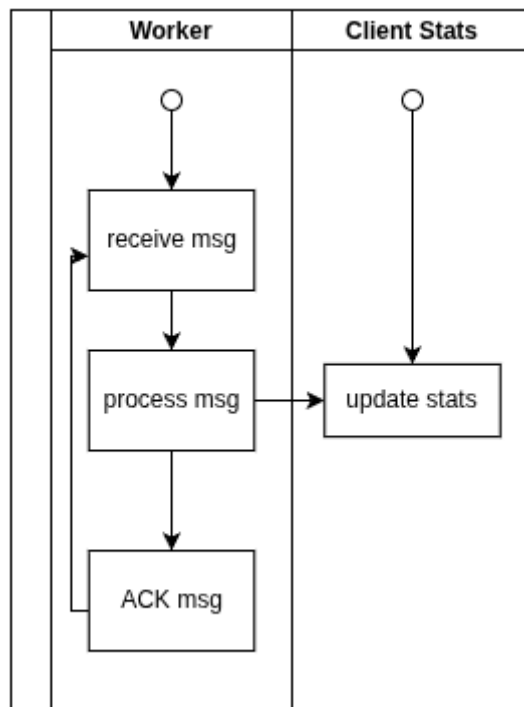
- **Worker:** un nodo de procesamiento de tipo Filter, Group o Join.
- **Client Stats:** entidad que almacena estadísticas de un cliente.
- **Partial Results Queue:** cola de RabbitMQ efímera
 - Recibe los resultados parciales de los nodos del mismo tipo.
 - Luego se elimina.

Información contenida por cada tipo de mensaje³:

- Mensaje de datos
 - Id de cliente
 - Datos
- Mensaje de EOF
 - Id de cliente
 - Total de mensajes emitidos por etapa anterior
- Mensaje de consulta de resultados parciales
 - Id de cliente
 - Nombre de cola
 - En donde debe enviar respuestas
- Mensaje de respuesta de resultados parciales
 - Id de cliente
 - Datos
 - Estadísticas de cliente
 - Cantidad de mensajes procesados y emitidos

→ Flujo de un mensaje que contiene datos a procesar por un nodo.

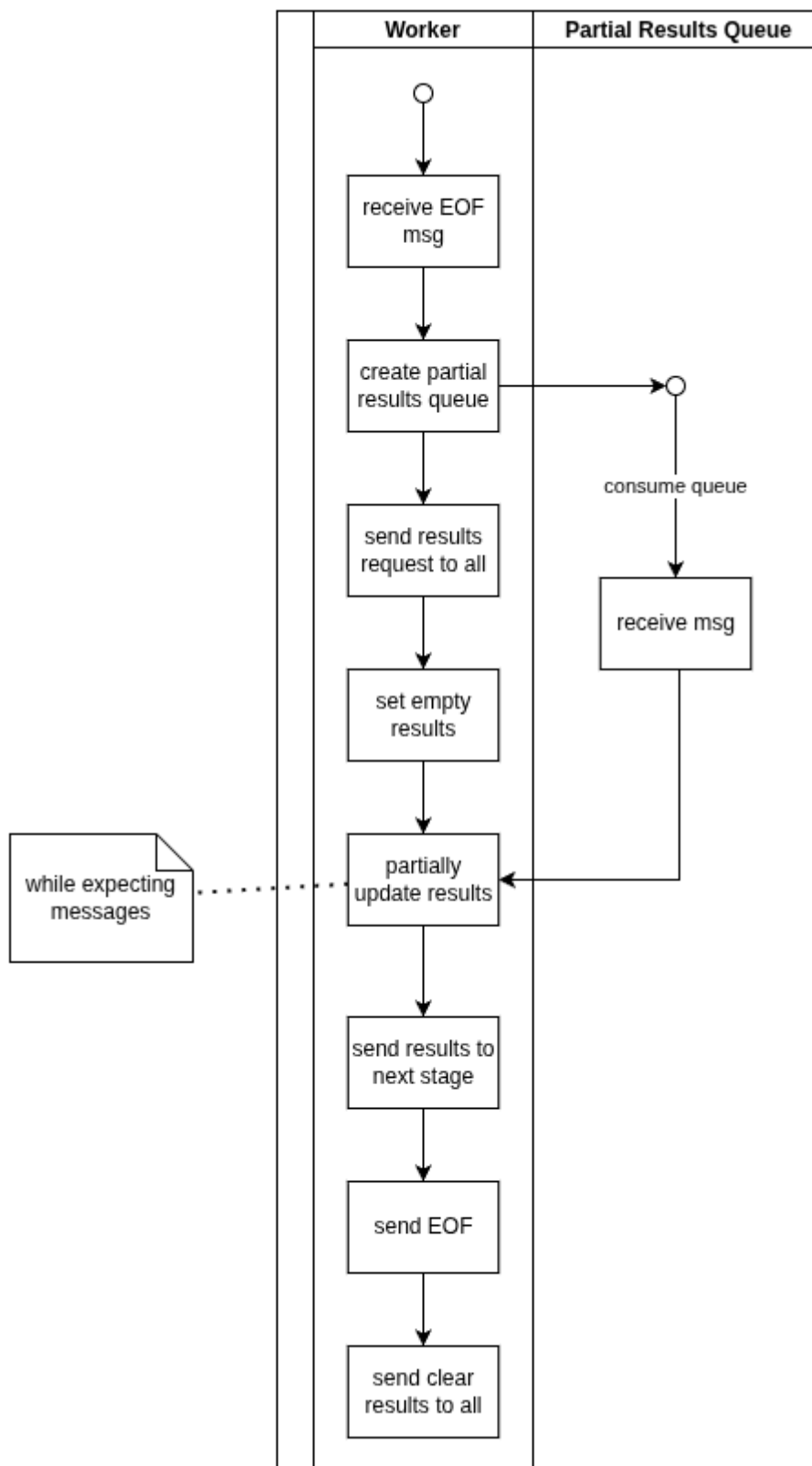
³ Solamente se mencionan los campos relevantes para los diagramas presentes.



Este es el flujo de la mayoría de los mensajes que circulan por los nodos.

Un mensaje es recibido, se procesa, se actualizan las estadísticas del cliente dueño del mensaje y se envía un ACK.

→ Flujo de un mensaje de EOF. No contiene datos.



Cuando se recibe un mensaje de tipo EOF, se inicia un flujo especial para consensuar resultados parciales entre todos los nodos del mismo tipo. Necesitamos asegurar que

todos estos nodos hayan terminado de procesar los datos del cliente del cual recibimos el EOF.

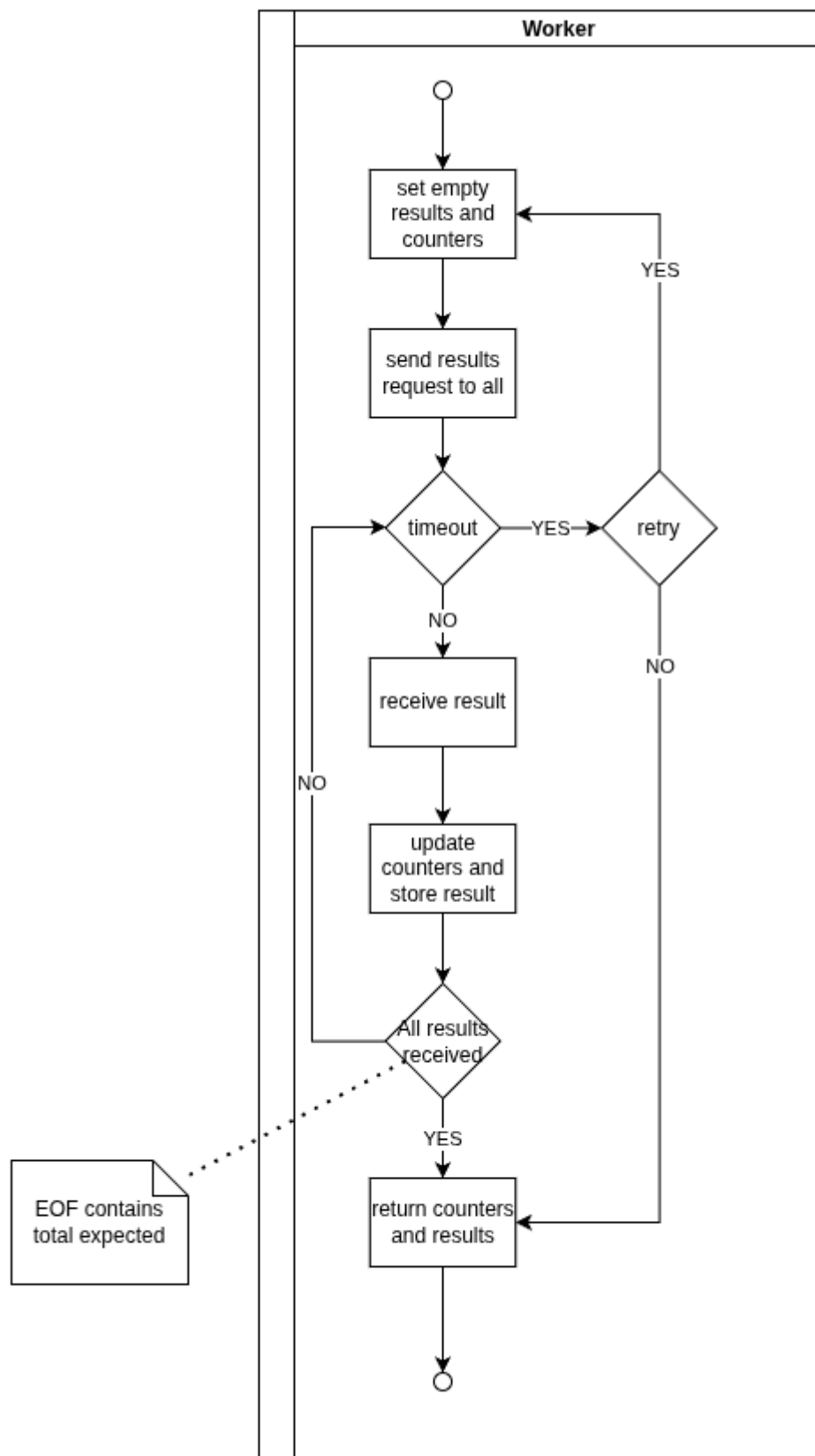
Para lograrlo, primero debemos crear una cola en la cual recibiremos los resultados parciales de todos ellos. En el punto siguiente *-Flujo de Partial Results Queue-* se detalla cómo procesa los mensajes entrantes.

Luego, envíamos un mensaje de tipo “request” a todos estos nodos para notificarles que estamos consensuando resultados. En el punto *-Flujo de “request” de resultados parciales-* se detalla cómo se procesan esos mensajes.

Una vez enviada esa request, esperamos a que todos los resultados parciales lleguen, juntándolos con los que ya fuimos recolectando.

Al finalizar, enviamos los resultados a la próxima etapa, propagamos el EOF actualizando la cantidad de mensajes que emitimos y enviamos a los nodos un mensaje para indicarles que pueden liberar los datos de este cliente (ya no se van a necesitar).

→ Flujo de Partial Results Queue



Cuando declaramos la cola para recibir resultados, entramos en este flujo.

Se inicializan los resultados como vacíos y enviamos un mensaje de broadcast a todos los nodos (incluido el mismo nodo que inicia el proceso) para pedir los resultados parciales.

Vamos a recibir mensajes de todos los nodos, actualizando resultados parciales y estadísticas.

Por cada mensaje recibido, me fijo si ya recibí todo chequeando que la cantidad de procesados (proveniente de la suma de las estadísticas) haya alcanzado la cantidad de emitidos que indica el mensaje de EOF.

En caso de tener todos los resultados, los retorno y finalizo.

En caso de no tener todos los resultados, hay un timeout corriendo por cada mensaje recibido. Si no recibo un nuevo mensaje en X tiempo, vuelvo al inicio y reintento.

Los reintentos se realizan una cantidad limitada de veces para evitar frenar todo el procesamiento, aunque los resultados finales pueden estar corruptos⁴.

→ Flujo de “request” de resultados parciales

Cada nodo está escuchando siempre por consultas de resultados parciales y responde siempre.

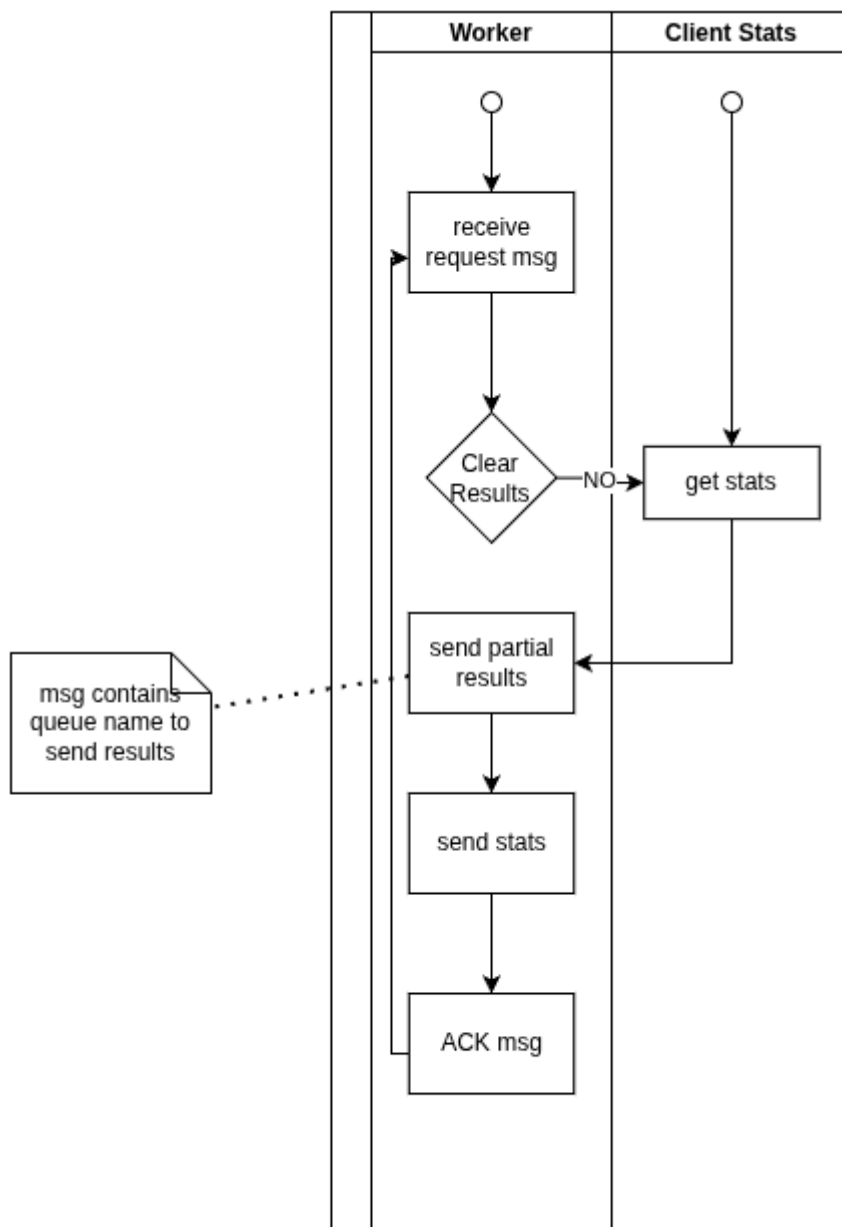
Hay dos caminos:

- Le piden datos
- Le avisan que se puede limpiar datos, es decir, liberar memoria

Idealmente, suceden ambos y en ese orden.

- Si es una consulta de datos:

⁴ Todavía no tenemos una forma de indicar que hubo timeout y los resultados no son correctos. En esta instancia podemos determinarlo mirando los logs.



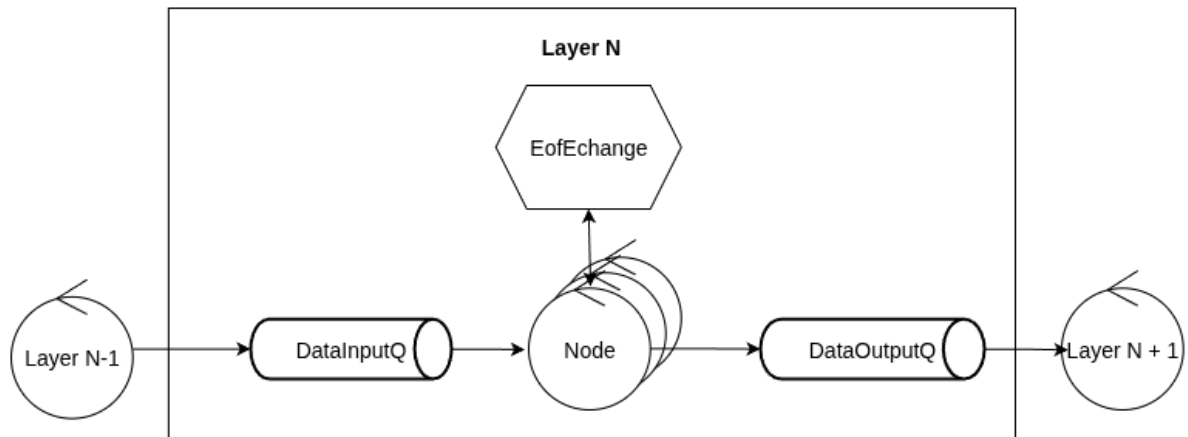
Del mensaje toma el nombre de la cola donde depositar los resultados. Los envía, envía luego las estadísticas del cliente y finaliza con un ACK.

- Si es un aviso para poder limpiar memoria:

En este caso, solamente elimina los resultados acumulados para el cliente.

Es necesario entender que este mensaje solamente se recibe cuando un nodo asegura haber enviado toda la información a la próxima etapa con éxito.

Independencia entre capas

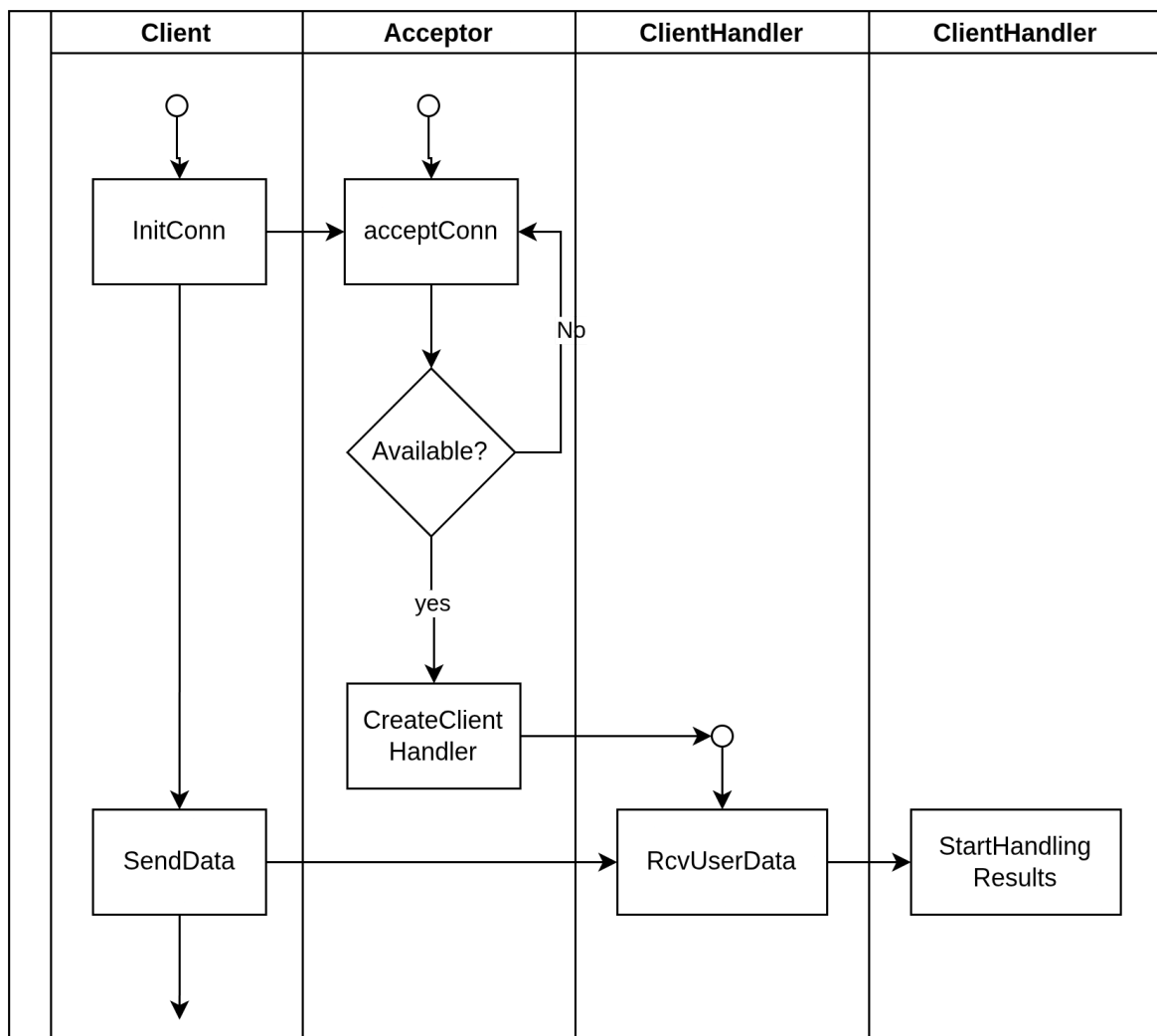


Con este último diagrama queremos remarcar que cada etapa es independiente de las demás. El único dato que requieren de la etapa anterior es la cantidad de mensajes emitidos para así contrastar con la cantidad de mensajes procesados.

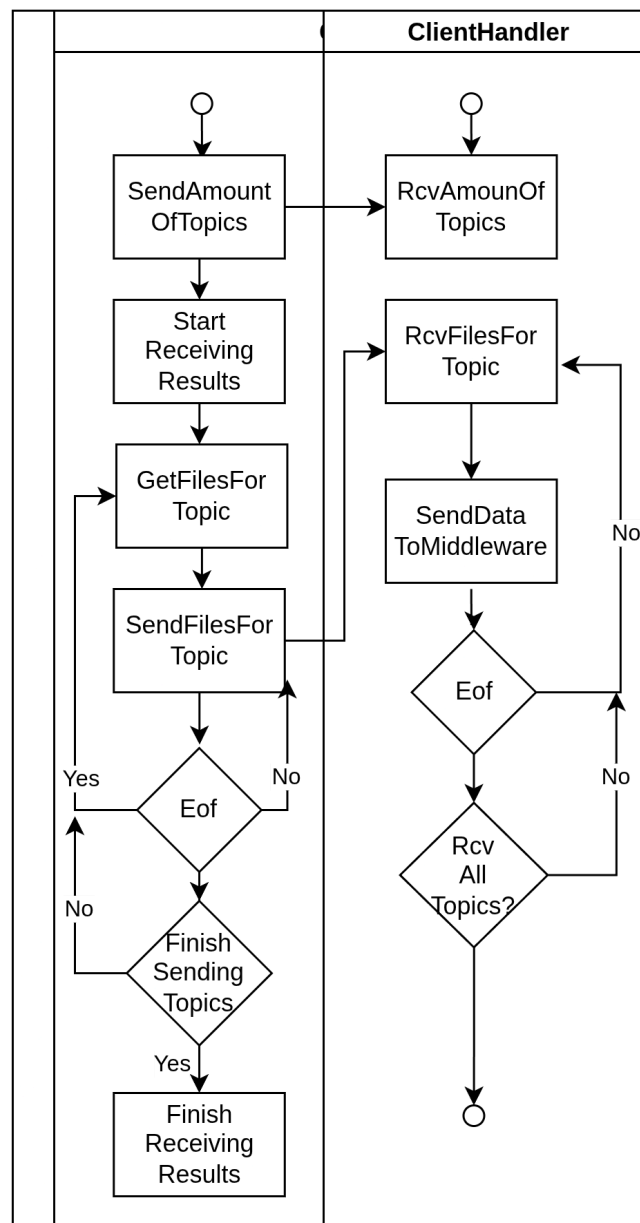
Sin embargo, las otras etapas no necesitan saber cuántos nodos tiene la etapa actual.

Envío de datos del cliente

Respecto al envío de datos por parte del cliente, inicialmente intenta conectarse con el ClientHandler, este le otorga la conexión solamente si tiene capacidad disponible, esta cantidad máxima de clientes es configurable.



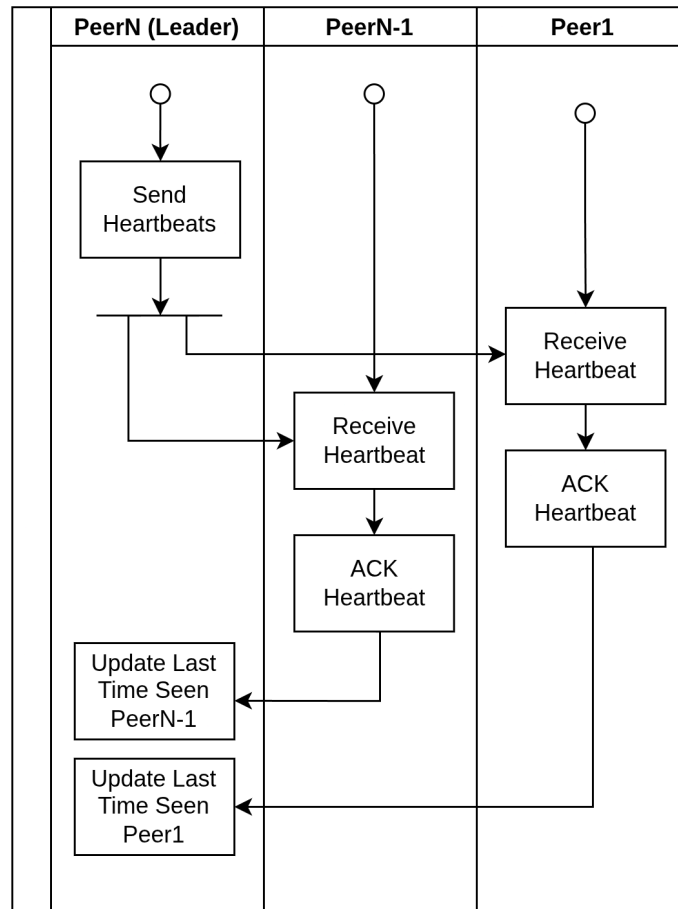
Una vez aceptada la conexión, comienza el envío de datos por parte del cliente. Esto se lleva a cabo de forma anidada por tipo de dato: primero se escanean los tipos de datos configurados y se buscan las carpetas correspondientes (en el cliente llamado “topic”), se envía la cantidad de topics al ClientHandler y se procede a enviar los archivos para un determinado topic, por ejemplo un CSV de transactions.



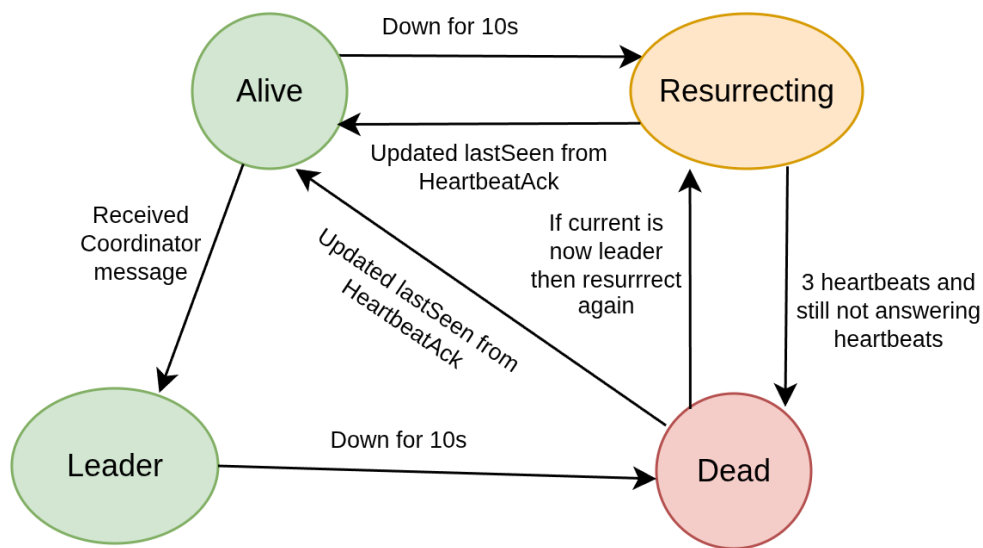
El procesamiento de resultados se realiza de manera asíncrona tanto para el cliente recibiendo del ClientHandler como para el ClientHandler sacando resultados de la queue de resultados para el cliente en cuestión.

tiempo, establecida inicialmente en 10 segundos. Esta ventana permite la pérdida en la red de hasta 3 mensajes heartbeat (tanto el heartbeat mismo como su ACK), dado que se envían heartbeats cada 2 segundos. Una vez detectado como caído con este mecanismo de red, el líder procede a resucitar dicho nodo ejecutando un comando de docker (ya que la imagen del container utiliza DockerInDocker para poder comunicarse con el docker daemon del host).

Una vez emitidos los heartbeats, los ACKs son recibidos de forma asíncrona, actualizando la última vez visto a medida que llegan.

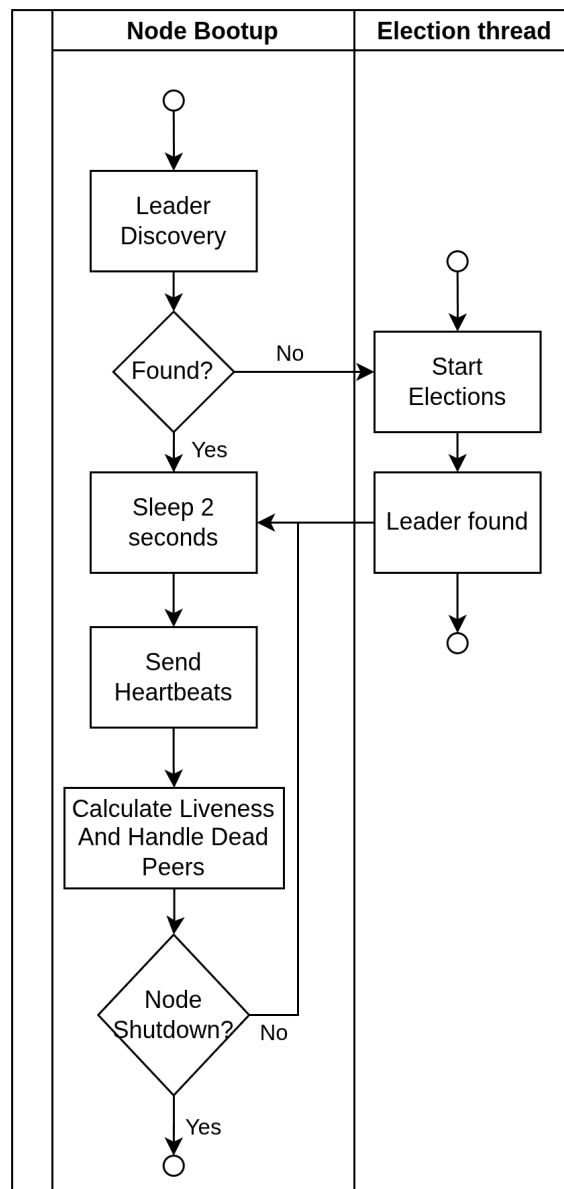


La forma en la cual el líder trackea el estado de los peers es mediante una máquina de estado asociada a cada peer:

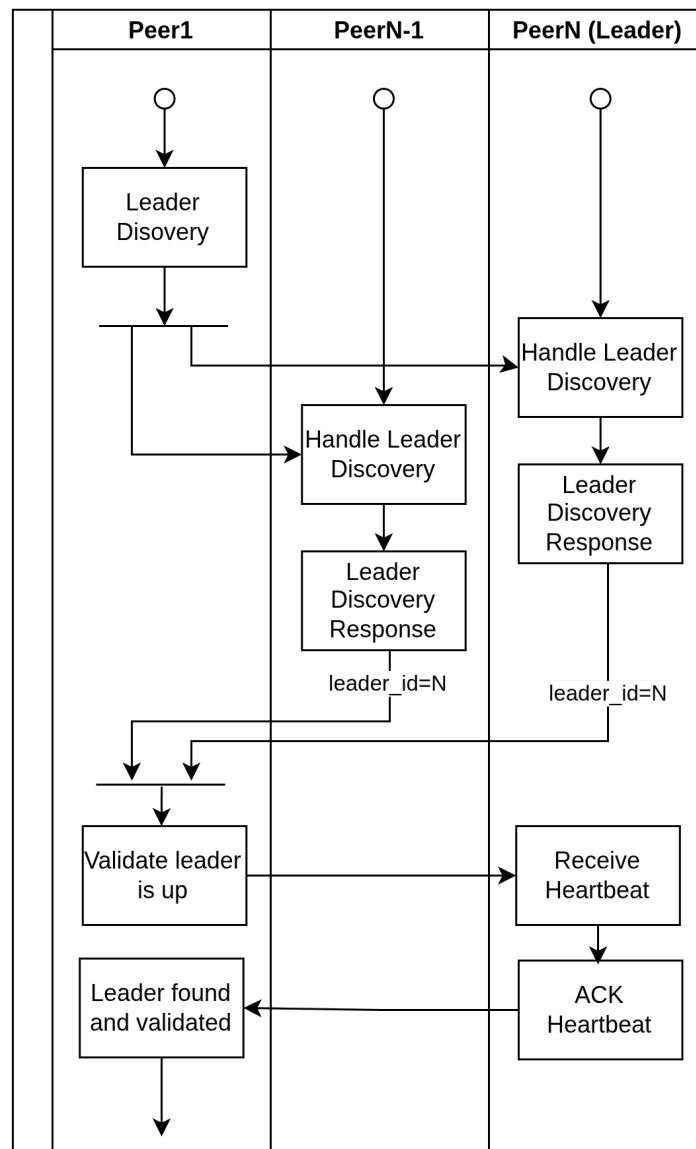


En este caso, el estado “Leader” se reserva para el tracking desde los peers hacia los demás peers, ya que un peer puede cambiar el estado de otro peer a Leader cuando le hayan llegado los mensajes correspondientes, sin embargo los peers no tienen forma de saber que otro peer (que no sea el líder) está caído o reviviendo. Esto es porque los peers sólo envían heartbeats al líder, no entre sí.

La ejecución general del WatchMesh se puede resumir en el siguiente diagrama, donde la parte del manejo de los peers caídos difiere dependiendo de si el nodo es un líder o no. Si el nodo es un líder entonces revivirá los nodos caídos. Si el nodo no es líder entonces comenzará un ciclo de elecciones para elegir un nuevo líder.

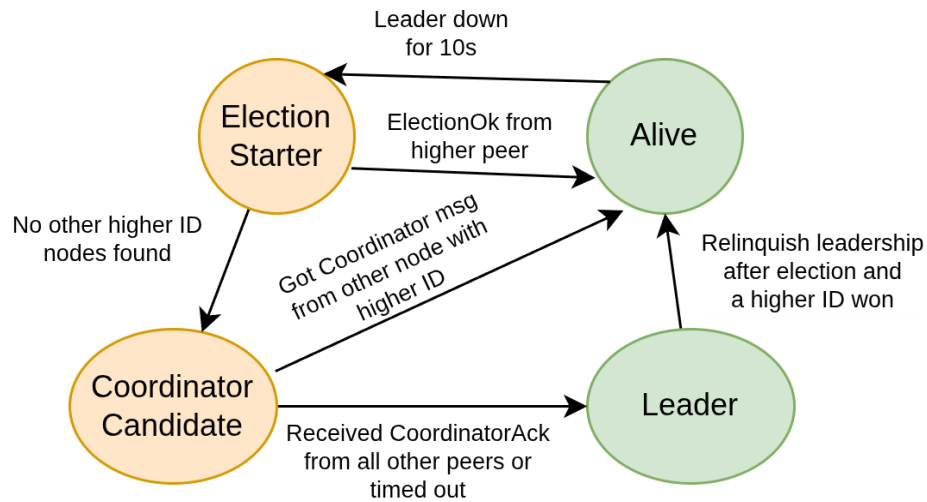


La primera etapa consiste en realizar un LeaderDiscovery, esto consiste en consultar a los demás nodos quién es el líder, de esta manera un nodo que recién arranca puede acoplarse rápidamente al sistema actual, minimizando mensajes. Este proceso se ve en el siguiente diagrama:



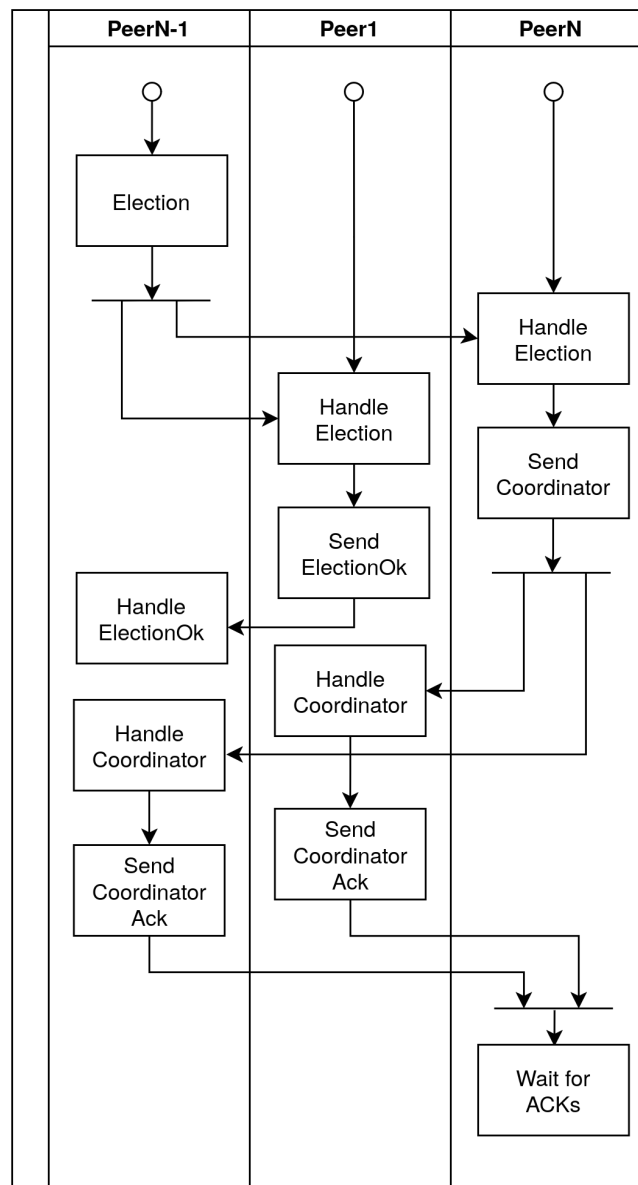
Si se perdieran los mensajes de LeaderDiscovery eventualmente se comenzaría un proceso de elección.

Los nodos también llevan el registro de su propio estado para con el resto del sistema, de esta manera se puede minimizar la cantidad de mensajes en un ciclo de elecciones. Esto se lleva a cabo con la siguiente máquina de estado:



El proceso de elecciones se hace, igual que todas las comunicaciones del WatchMesh mediante UDP. Se utiliza el algoritmo Bully para determinar quién gana las elecciones. De los mensajes disponibles de la elección (Election, ElectionOk, Coordinator, CoordinatorACK) se pueden perder los mensajes Election y ElectionOk, esto es porque eventualmente se iniciaría otra elección o alguien se proclamaría líder y enviaría su Coordinator. Para asegurar que llega a toda costa el mensaje Coordinator a todos los nodos, se espera que todos los demás nodos envíen su CoordinatorACK, o hagan timeout (porque están caídos). De esta manera hay consistencia y todos los nodos tienen el mismo líder.

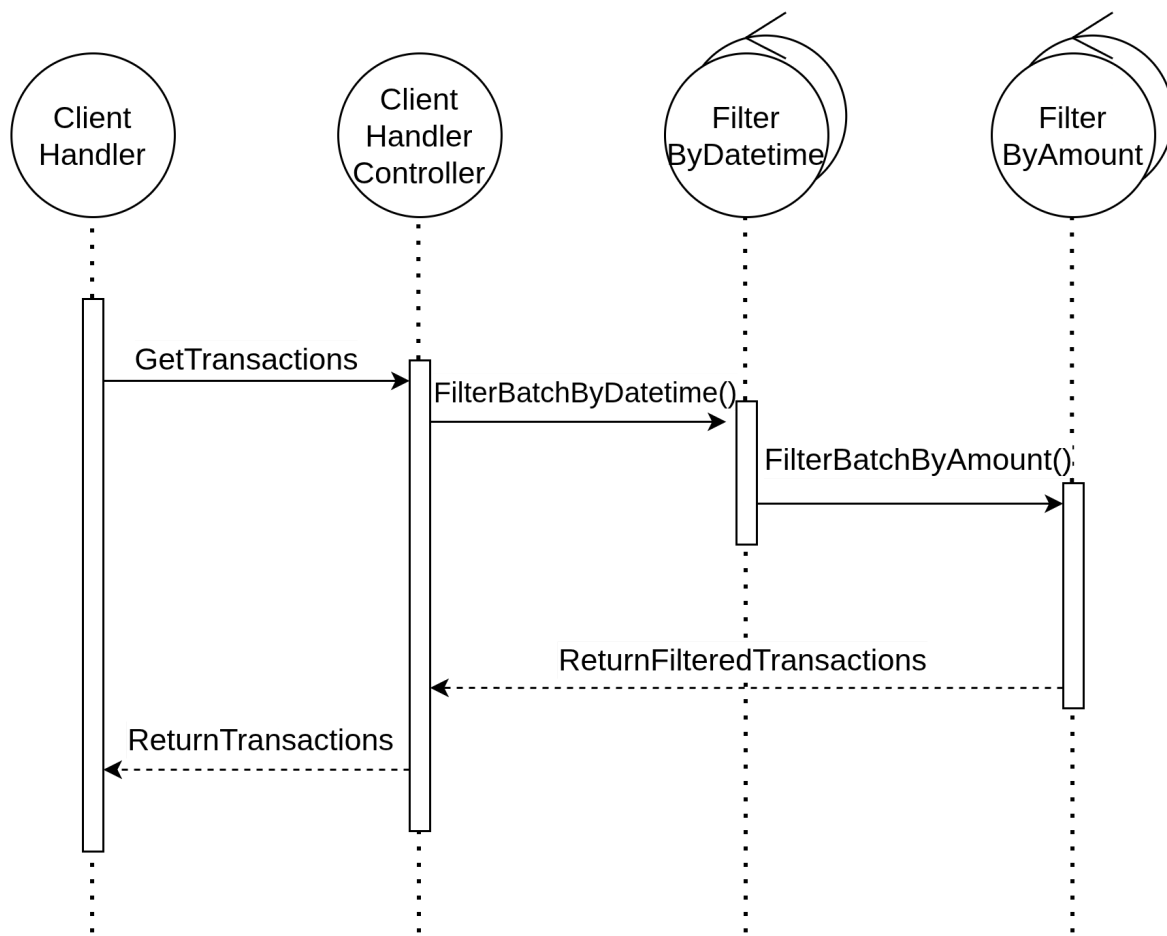
Se presenta un diagrama de un proceso de elección:



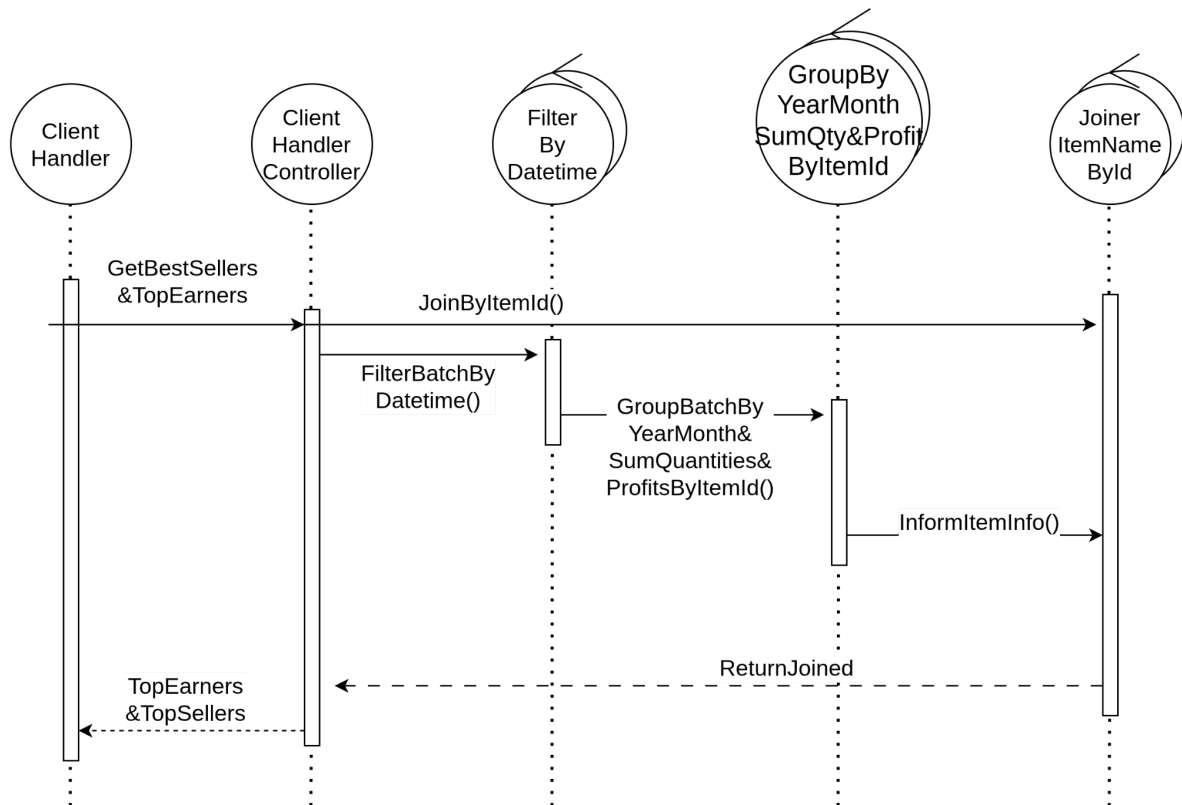
Diagramas de secuencia

Queries

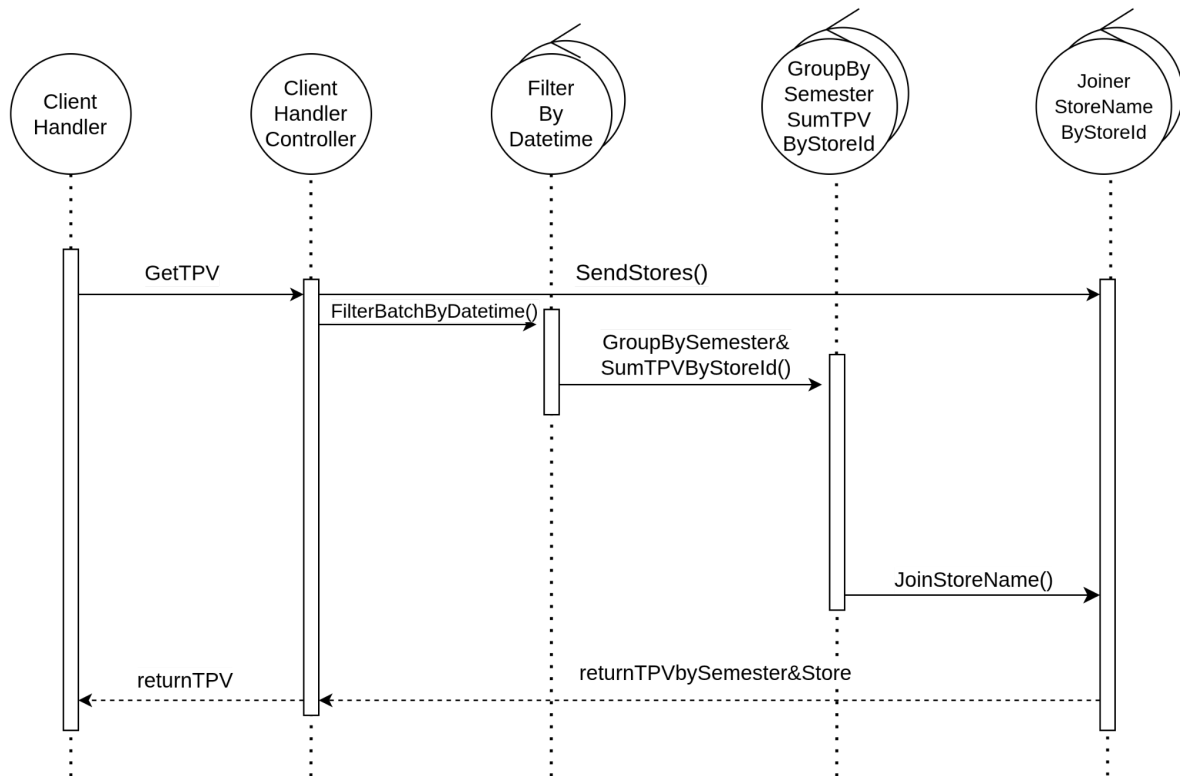
A continuación, se presentan los diagramas de secuencia que muestran el flujo y la interacción de los nodos para llevar a cabo el desarrollo de todas las operaciones:



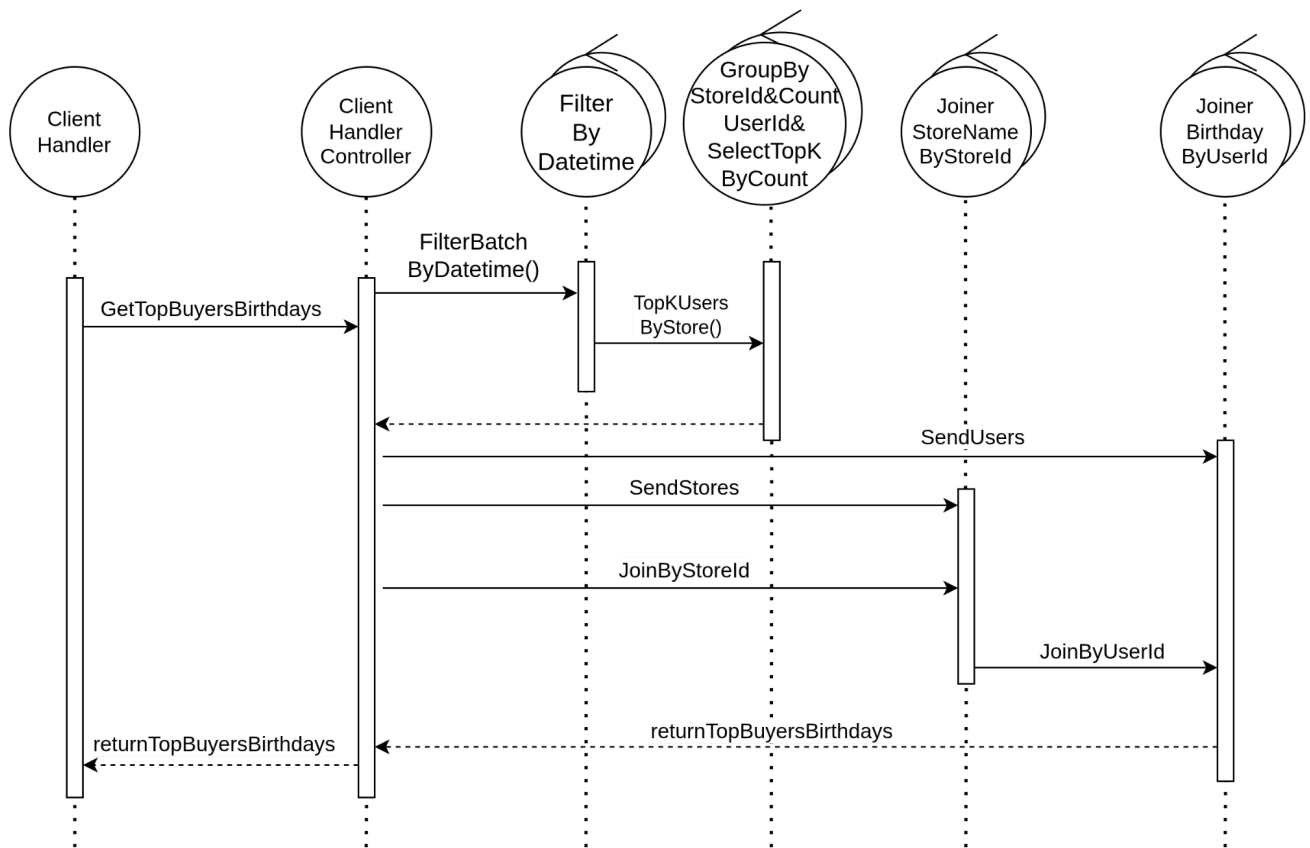
Consulta 1



Consulta 2



Consulta 3



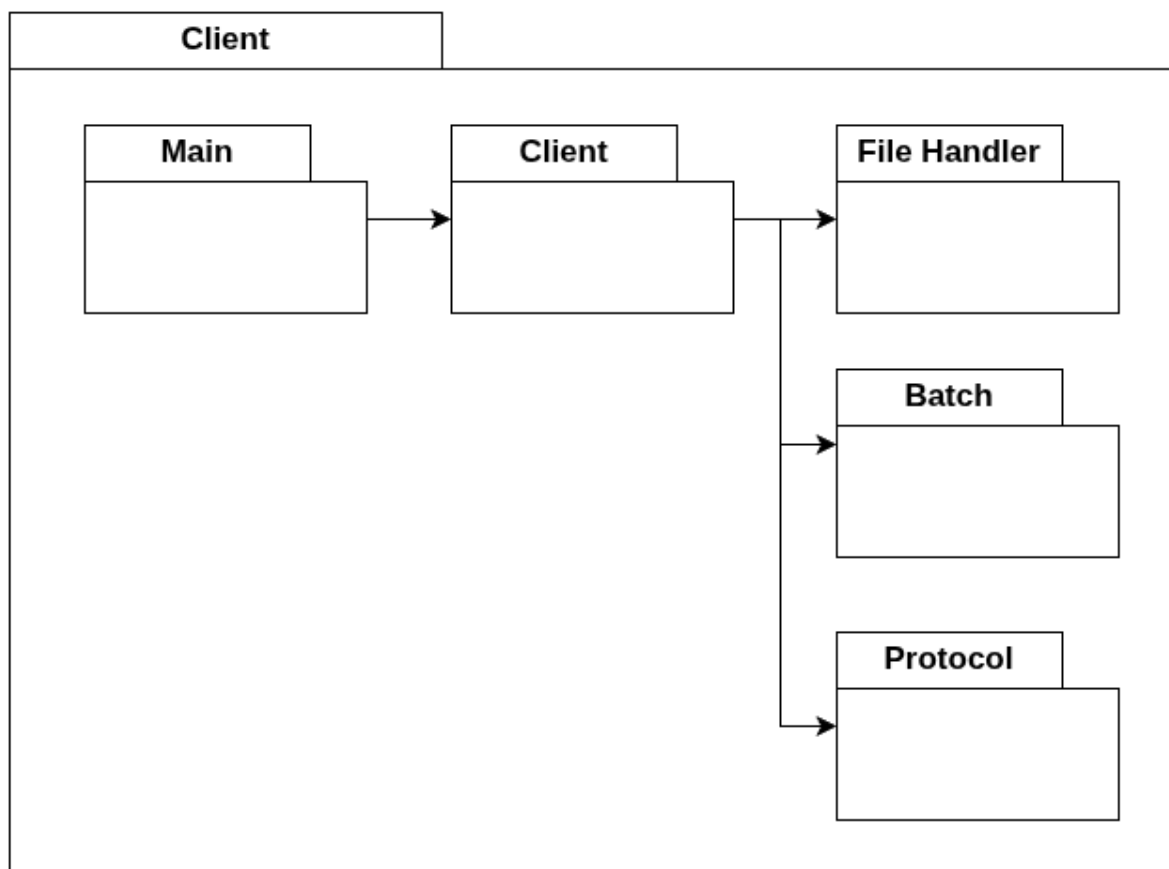
Consulta 4

5. Vista de desarrollo

Diagrama de paquetes

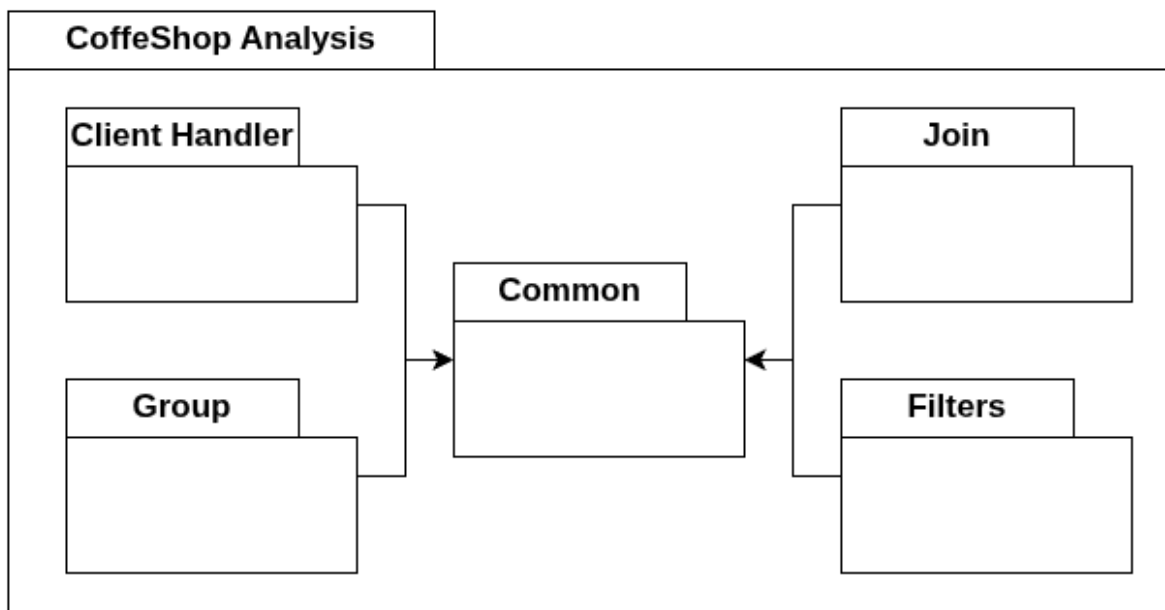
Los paquetes requeridos para desarrollar el proyecto se pueden dividir en dos grandes grupos:

En un principio se tiene el cliente, externo a nuestro sistema distribuido el cual se encarga de subir la información necesaria para poder realizar las consultas:

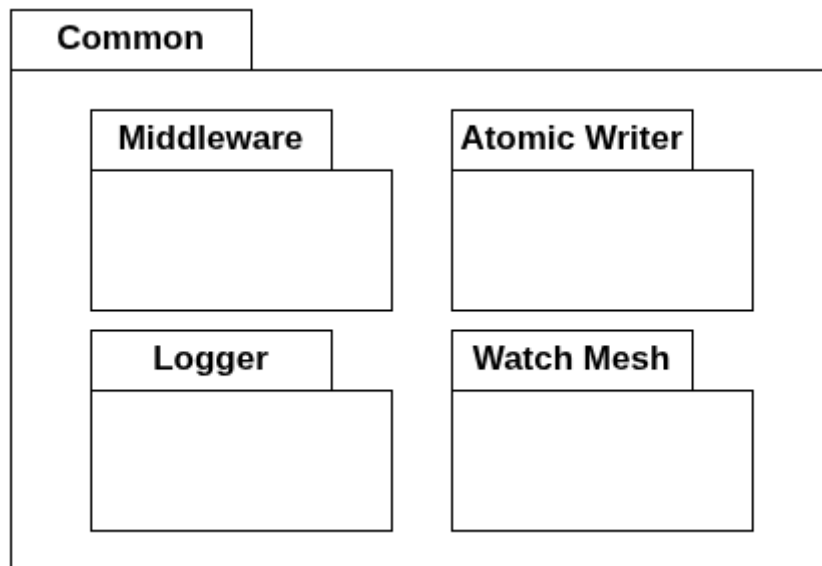


- *Main*: punto de inicio
 - *Client*: mantiene todo lo necesario para iniciar el proceso desde el lado del cliente
 - *File Handler*: helper para leer archivos
 - *Batch*: helper para crear batches
 - *Protocol*: contiene el protocolo de conexión con el servidor

Y luego se tiene el diagrama de paquetes correspondiente al sistema distribuido:

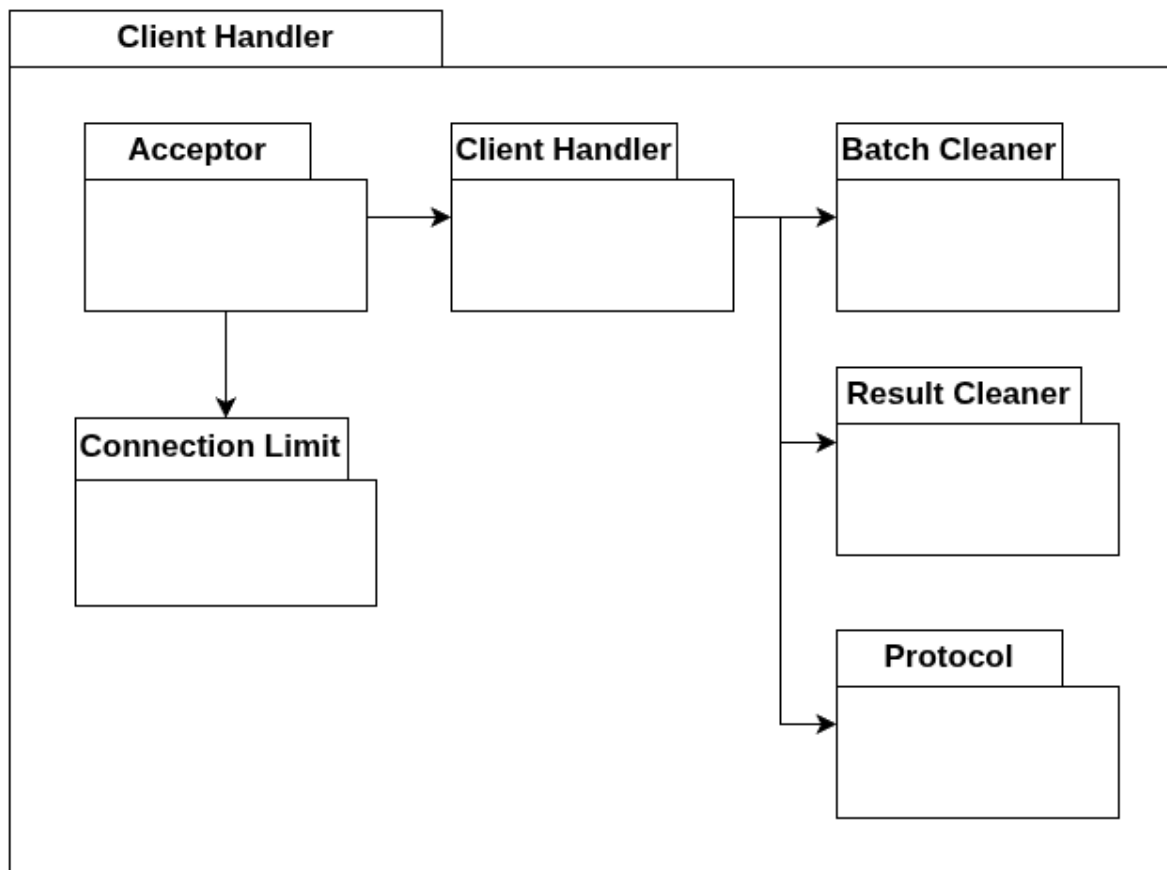


Common contiene paquetes útiles usados por el resto de entidades.



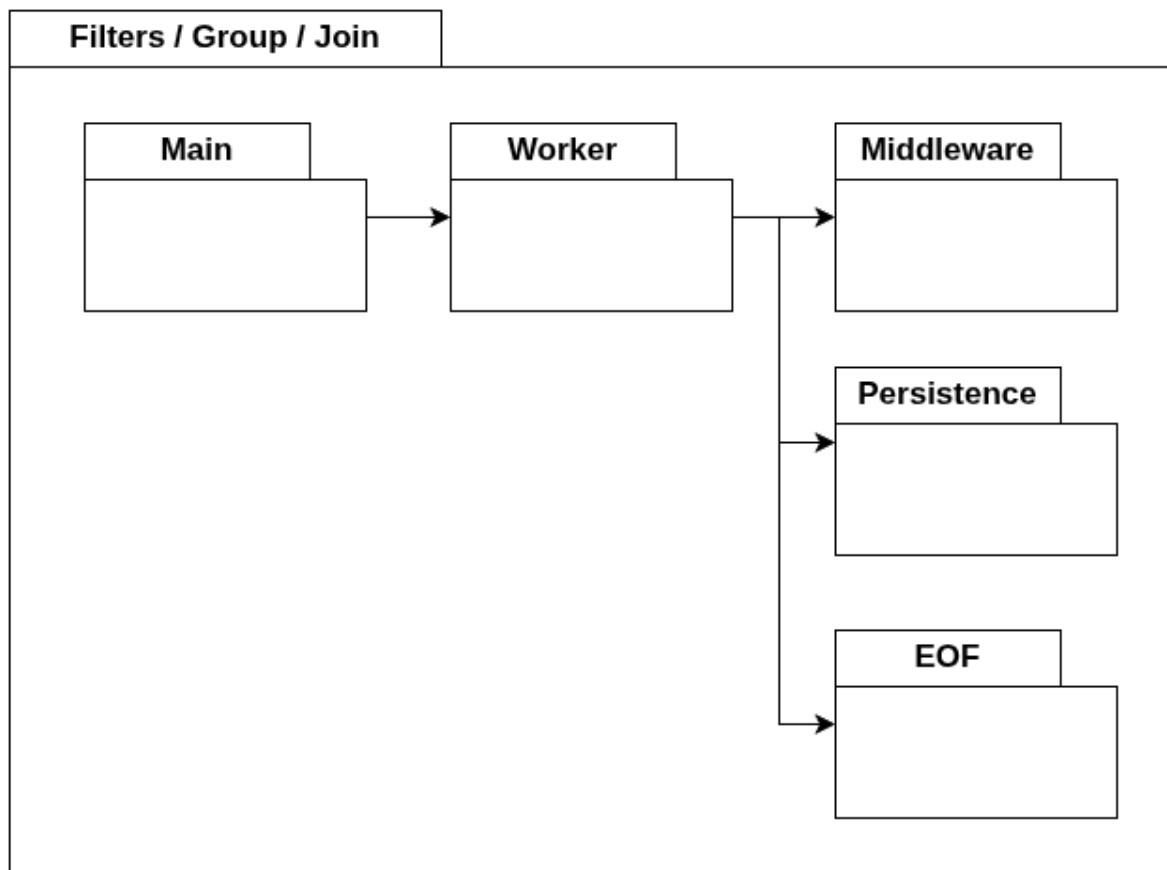
- *Middleware*: código relacionado a configurar y personalizar la conexión con RabbitMQ.
- *Logger*: código que permite imprimir logs personalizados.
- *Atomic Writer*: código que permite escribir archivos de forma atómica.
- *Watch Mesh*: código que permite el monitoreo y resurrección ante caídas entre nodos.

ClientHandler define las conexiones con nuevos clientes y empieza el procesamiento de datos.



- *Acceptor*: acepta nuevas conexiones
 - *Connection Limit*: permite limitar la cantidad de conexiones en simultáneo.
 - *Client Handler*: procesa una conexión particular
 - *Batch Cleaner*: elimina datos innecesarios enviados por la conexión.
 - *Result Cleaner*: elimina datos innecesarios en la respuesta al cliente.
 - *Protocol*: contiene el protocolo de conexión con el cliente.

Filters / Group / Join son los nodos de procesamiento.



- *Main*: punto de inicio. Lee la configuración
 - *Worker*: código para iniciar el worker
 - *Middleware*: código del Worker relacionado con la conexión a RabbitMQ
 - *Persistence*: código relacionado a la persistencia de datos con Atomic Writer
 - *EOF*: código relacionado al consenso de EOF

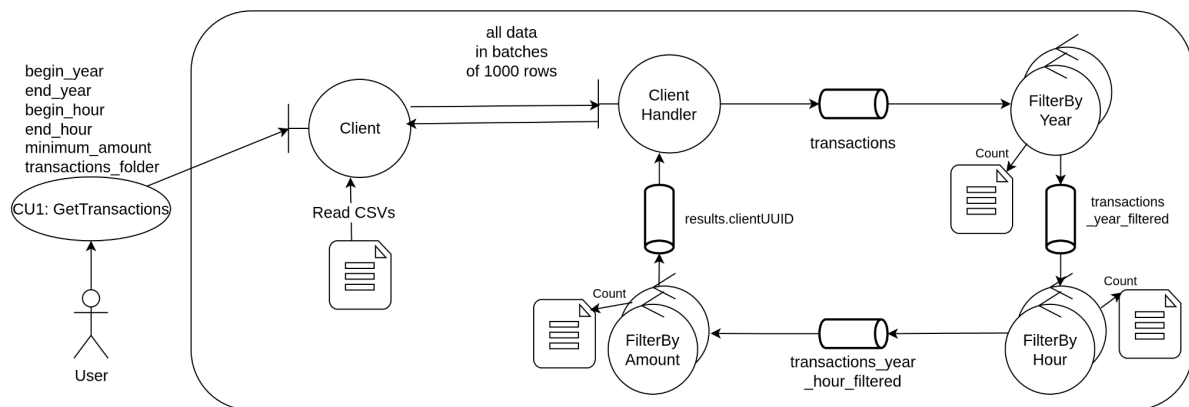
6. Vista Física

Diagrama de robustez

El cliente interactúa con el boundary ClientHandler del lado del servidor, por el cual se conecta mediante un socket TCP para el envío de datos en batches de una cantidad configurable de líneas.

Una vez llegada toda la data al ClientHandler, este envía la información adecuada mediante el middleware a las queues de input de cada query. Una vez terminado el procesamiento de las queries, cada uno deposita el output en una queue de output para un dado Client (identificado por su clientID) del cual el ClientHandler sacará los resultados finales para enviar al Client mediante el socket TCP.

Query 1

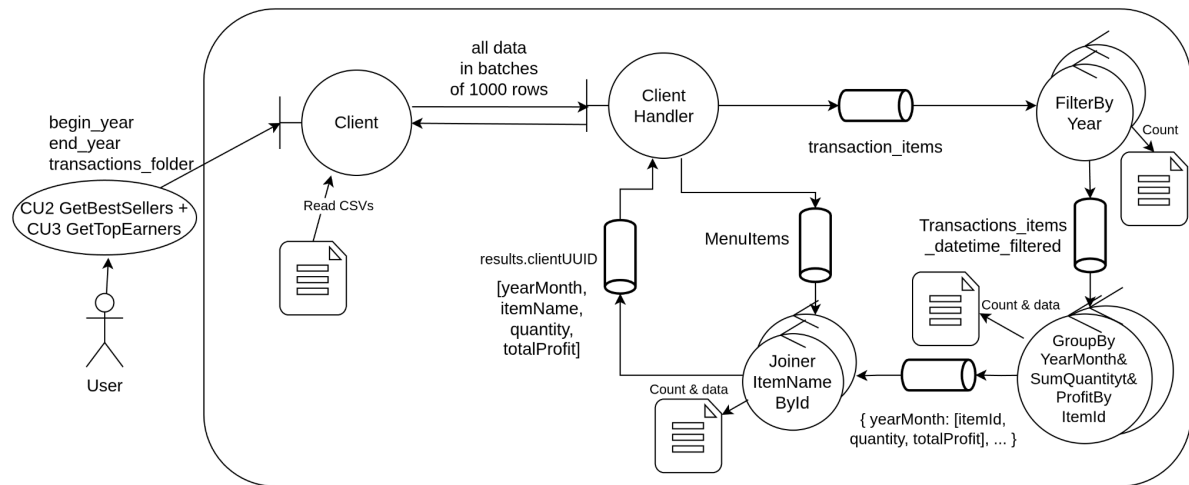


En este caso, todas las transacciones se insertan en batches en una cola que será consumida por un filtro por año. Las transacciones de cada batch que cumpla, se inserta en la cola del filtro por hora.

El filtro por hora repite lo mismo pero con su función de filtro e inserta en una próxima cola, filtro por monto.

Por último, las transacciones que pasen el filtro por monto se envían a una cola de resultados (identificada por el clientID) que es consumida por el ClientHandler.

Query 2



Los ítems de transacciones se insertan en la misma cola de filtro por año, pero van identificados como un tipo de dato distinto, por lo tanto sus resultados terminan en otra cola, para ser consumidos por un tipo de nodo group.

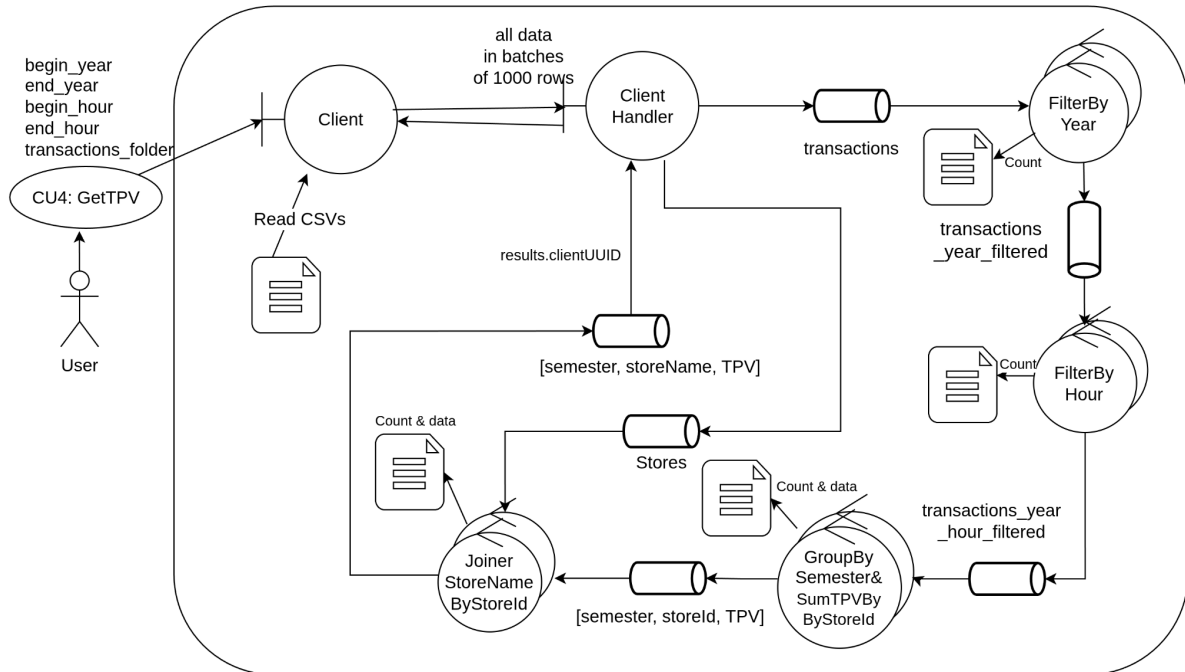
El nodo group tomará estos ítems, asociandolos por “año-mes” y reduciendolos por cantidad y monto, de modo que si un ítem se repite, se agrupan y se suman.

Luego, se envían a una cola consumida por un nodo de tipo join.

Este join contiene una tabla auxiliar, previamente recibida en una cola especial, que relaciona id de ítem con su nombre. Así, a los resultados del group, se le reemplaza el id del ítem por el nombre.

Por último, se envían los resultados al client handler.

Query 3



Comparte las primera dos etapas con el flujo de la Query 1.

Luego, existe otro tipo de nodo group que consume los mensajes de la cola de resultados del filtro por hora.

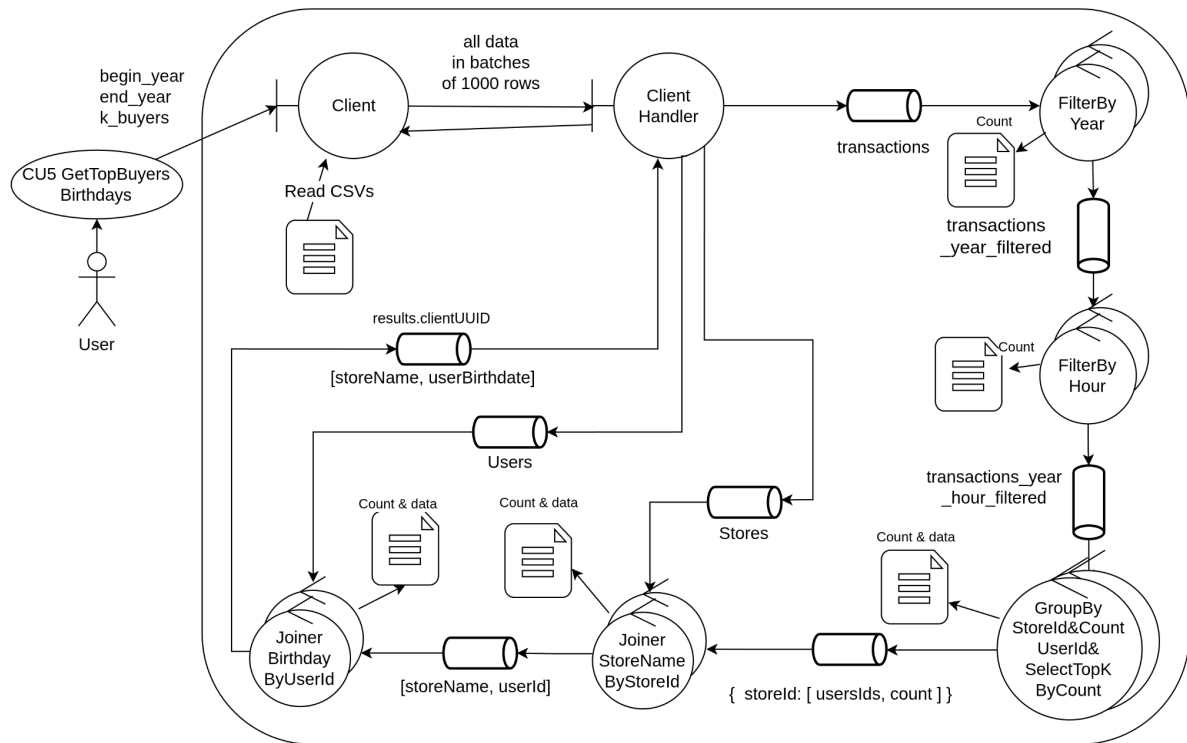
Este group toma las transacciones y las agrupa por semestre, reduciendo por tienda (store) y sumando el subtotal.

Así, para cada semestre, hay un subtotal recaudado por tienda.

Luego, similar a la query anterior, hay un nodo join que consume estos resultados e inserta el nombre de la tienda. También cuenta con una tabla auxiliar para esto.

Finalmente, se envían los resultados finales.

Query 4



De nuevo se comparten las primeras dos etapas de filtro de la query 1, sin duplicar los mensajes.

Luego, un nodo group tomará los resultados, agrupándolos por tienda y contando la cantidad de apariciones de cada cliente en cada una de ellas.

Una vez se tienen todas las transacciones, nos quedamos con los K compradores con más transacciones realizadas.

Por último, existen dos nodos de tipo join que insertarán el nombre de la tienda primero, y la fecha de cumpleaños de cada cliente luego.

Observación: como la tabla auxiliar de usuarios es considerablemente grande en comparación a las anteriores, no se la guarda en memoria. Lo que se hace es esperar a los resultados del join anterior y por cada batch de usuarios, se insertan los que correspondan.

Esto funciona porque los resultados son lo suficientemente pequeños como para mantenerlos en memoria mientras se leen los usuarios.

Finalmente, los resultados se envían al client handler.

Proceso de consenso para batch terminado

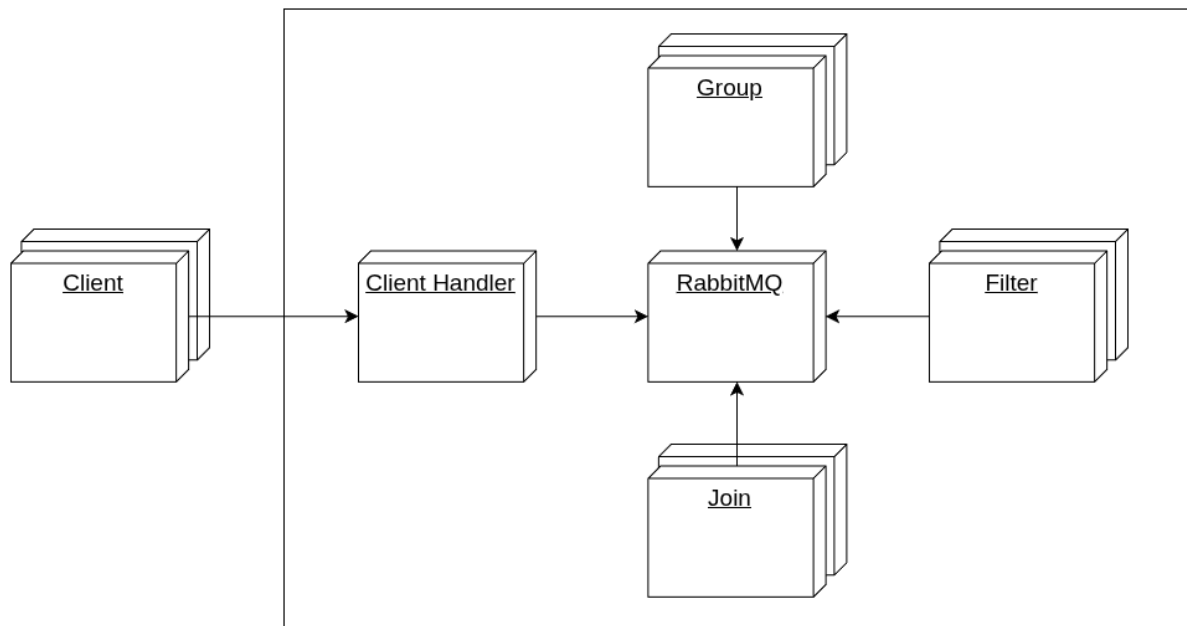
Para cada tipo de nodo es posible contar con una cantidad arbitraria de réplicas. Por esto, es necesario que los nodos se comuniquen entre sí para determinar cuándo han llegado al final de procesamiento.

Luego de enviar todos los batches, existe un mensaje de EOF que se deberá propagar por todas las etapas.

Sin embargo, este mensaje solamente es recibido por un solo nodo de esas réplicas, entonces entra en juego el mecanismo mencionado en la sección “Esquema de manejo de EOF”.

Diagrama de despliegue

Se presenta el diagrama de despliegue del sistema distribuido, mostrando explícitamente las posibilidades de escalabilidad horizontal con multicomputing:



Podemos ver claramente qué nodos dependen de cuáles para su creación.

Los nodos de procesamiento Filter, Group y Join pueden tener 1 o más réplicas, que se monitorean entre sí ante caídas (Watch Mesh).

Client, Client Handler y RabbitMQ son considerados como único punto de falla, ya que no tienen réplicas ni son resilientes ante caídas.

Para Client Handler tenemos una propuesta, no implementada, que consiste en levantar varias instancias y reutilizar el Watch Mesh para que se monitorean. Las instancias adicionales NO aceptan conexiones, sino que solamente monitorean para poder revivir procesos caídos.

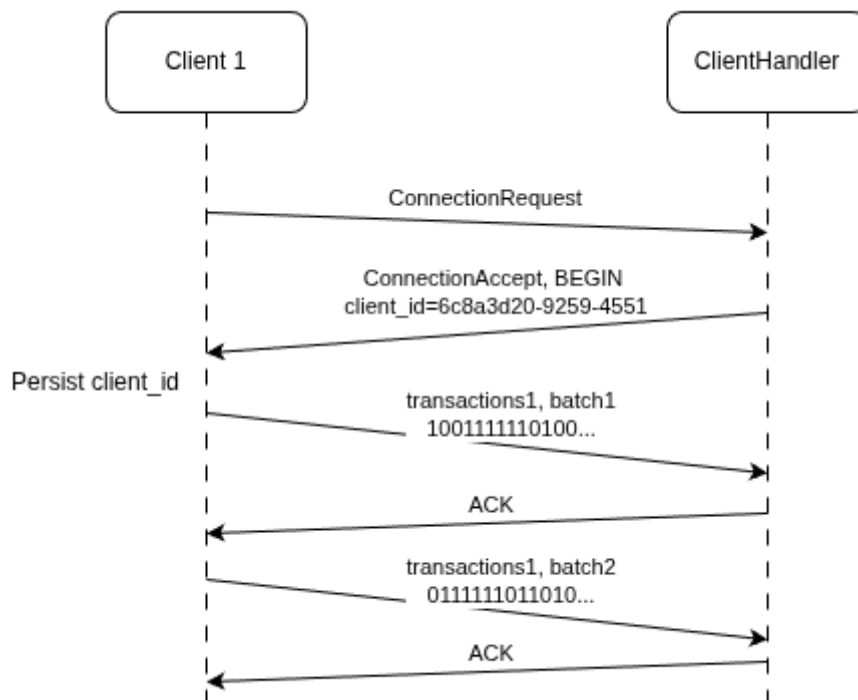
Client Handler puede manejar múltiples clientes a la vez. Los clientes todavía no pueden fallar durante el procesamiento y reconectarse, continuando desde el último estado válido. En la rama *feat/client-reconnection* del repositorio existe una implementación pero que no llegó a testearse por completo, por lo cual se propone como mejora.

El algoritmo para resiliencia del cliente consiste en guardar en disco el último estado válido enviado al servidor. Así, por cada batch enviado va a guardar el nombre del archivo que estaba enviando y el offset.

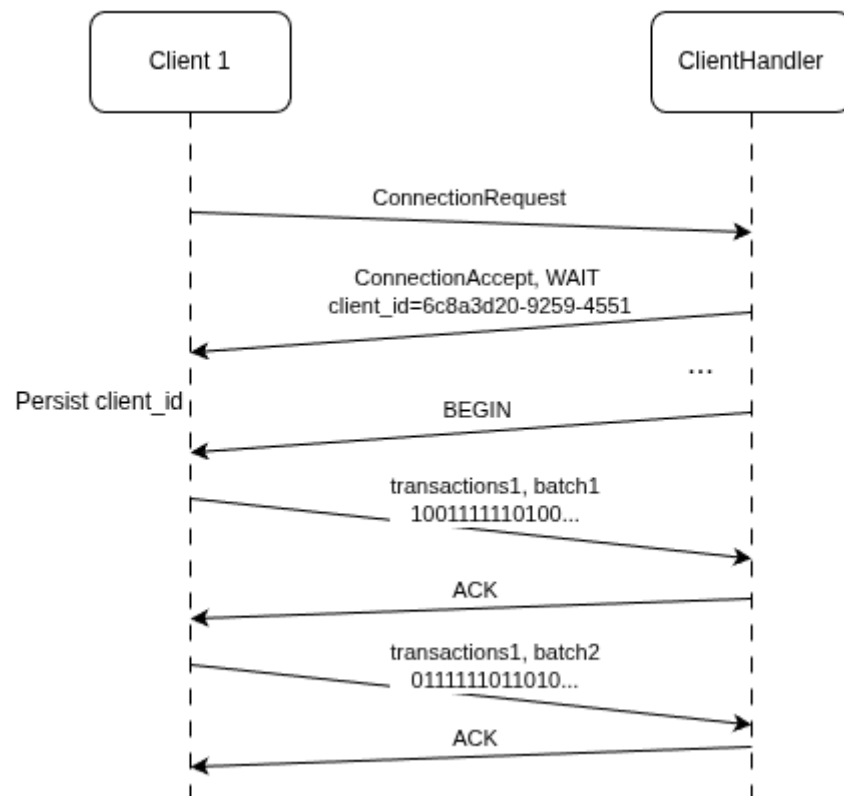
Si se cae, el Client Handler le “guarda” la sesión durante unos minutos y le admite reconectarse. En la reconexión, el cliente no envía todos los archivos desde cero, sino que continúa desde el último estado válido que llegó a guardar en disco.

Como el procesamiento de datos es idempotente, no corremos riesgo al duplicar algunos batches.

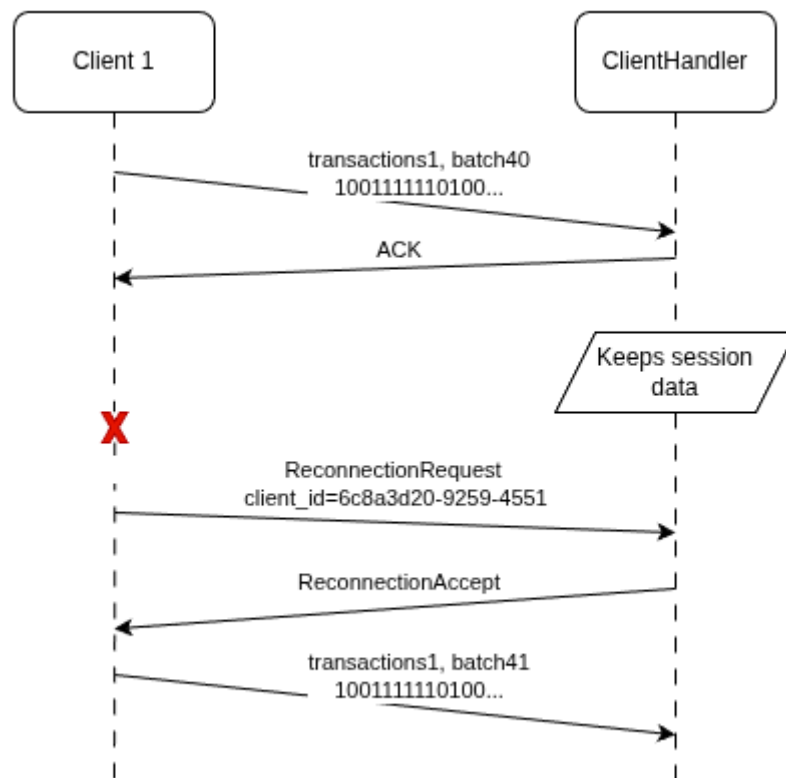
Una nueva conexión normal.



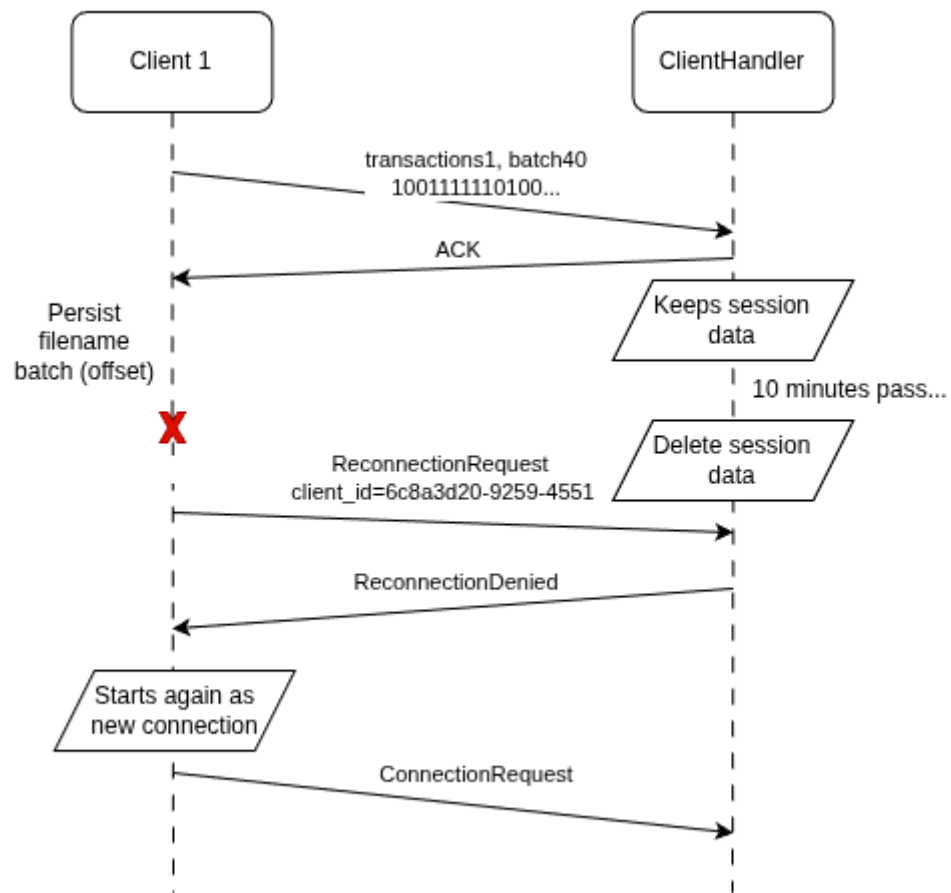
Una nueva conexión normal pero con el Client Handler con la capacidad de clientes al máximo, entonces pone en espera al nuevo cliente.



Reconexión de un cliente dentro de la ventana de tiempo permitida por el Client Handler.



Reconexión de un cliente fuera de la ventana de tiempo permitida. El Client Handler rechaza la reconexión y fuerza a iniciar una nueva desde cero.



7.División de tareas

Se provee una distribución de tareas de alto nivel para el desarrollo del proyecto:

Tareas	Integrante Responsable
Implementación del módulo de ingesta de datos en el cliente que lea y prepare mensajes (batch) para ser enviados al servidor	José
Implementación del ClientHandler y el routing de información para ejecutar todas las queries.	José
Implementación del pipeline de procesamiento para la consulta 1 que involucra: FilterByAmount y FilterByDatetime	Gabriel
Implementación de parte del pipeline de procesamiento para la consulta 2 que involucra: GroupByYearMonth y SumQuantityt&ProfitByYearMonth&itemId	Ramiro
Implementación de parte del pipeline de procesamiento para la consulta 2 que involucra: JoinerItemNameById	Gabriel
Implementación de parte del pipeline de procesamiento para la consulta 3 que involucra: GroupBySemester y SumTPVBySemester&StoreId	Gabriel
Implementación de parte del pipeline de procesamiento para la consulta 4 que involucra: GroupByStoreId , CountUserIdByStoreId y TopKSelectorByCount	José
Implementación de parte del pipeline de procesamiento para la consulta 4 que involucra: JoinerStoreNameByStoreId y JoinerBirthdayByUserId	Ramiro
Implementación de tolerancia básica de fallos en workers (reintentos por falla de conexión, graceful quit con SIGTERM)	Ramiro
Algoritmos de consenso, red de supervisión de salud de los nodos (Watch Mesh), soporte de fallos en el cliente	Gabriel
Simulación de casos de fallo en el código (Crasher), Idempotencia en los nodos, Caché	Ramiro

Escritura atómica de archivos (Atomic Writer), simulación de caos, test de integración, protocolo de recuperación	José
--	------

8.Apéndice: Tolerancia a fallos

En la siguiente sección se realiza una descripción de todos los elementos necesarios para generar un sistema con tolerancia a fallos.

Secciones de fallos

Se listan las circunstancias en las cuales se pueden presentar fallos en el sistema, de manera, que se establezca el listado de faltas posibles sobre el sistema

1. Post extracción de paquete de rabbitMQ
 - a. Antes de procesar el paquete: En caso de sacar un paquete de una cola de rabbit y presentar un fallo, simplemente rabbit reencolado el paquete
 - b. Post procesamiento de paquete: Si un paquete es procesado de manera exitosa pero no se llega a realizar un ack del paquete o procesar su información, simplemente se reencola el paquete
2. Pre grabar a disco
 - a. Antes de grabar a disco el resultado de la transacción: Si se llega a procesar un paquete con éxito pero durante su bajada a disco el nodo involucrado falla, el paquete se reencola para su futuro procesamiento
 - b. Post grabar a disco el resultado de la transacción: Una vez grabado el resultado del procesamiento de un paquete en disco, en caso de una caída, el protocolo de recuperación estará provisto de la información necesaria para poder identificar que el paquete reencolado fue previamente procesado
3. Pre realizar ACK en rabbitMQ
 - a. Post realizar ACK de un mensaje en rabbit: una vez procesado y posterior a hacer ACK el paquete habra sido sacado del sistema pero la información relacionada al mismo se encuentra en los datos grabados en memoria del nodo asociado a la transacción

Persistencia en el sistema

El sistema emplea la técnica atomic file replace, con el propósito de tener un sistema de control sobre los archivos. De esta forma, el sistema tiene la capacidad de escribir archivos y garantizar mediante la nomenclatura usada en los nombres que dichos archivos fueron escritos sin inconvenientes y no se encuentran en estados inválidos. Cada nodo contará con un volumen de docker con el propósito de simular un sistema de archivos independientes uno del otro que será visible dentro de una carpeta para el usuario.

En caso de que un fallo en el sistema ocurra durante la escritura de un archivo, dicho archivo tendrá un nombre temporal en la recuperación y podrá ser identificado como un archivo invalido.

Nodos Filtros

Los nodos filtros a pesar de prácticamente no tener estado, necesitan mantener el control sobre la transmisión del final del archivo reportado por el cliente, es por eso que, en cada nodo se almacena por cliente y tipo de archivo, la cuenta de la cantidad de los paquetes procesados. Por lo tanto, en caso de fallo en alguna de las capas de nodos filtros, se pueda realizar de manera confiable la transmisión del EOF a la siguiente etapa

Nodos Group

Los nodos group son nodos con estado que necesitan mantener almacenada la información desde el inicio de la transacción del cliente hasta el final, es decir, no pueden transmitir la información a la siguiente etapa sin antes haber realizado su operación sobre la totalidad de la información compartida por las capas anteriores. A excepción de los nodos del tipo TopK, la información almacenada es el resultado de haber agrupado el batch recibido con los datos existentes en el nodo al momento de recibir dicha información.

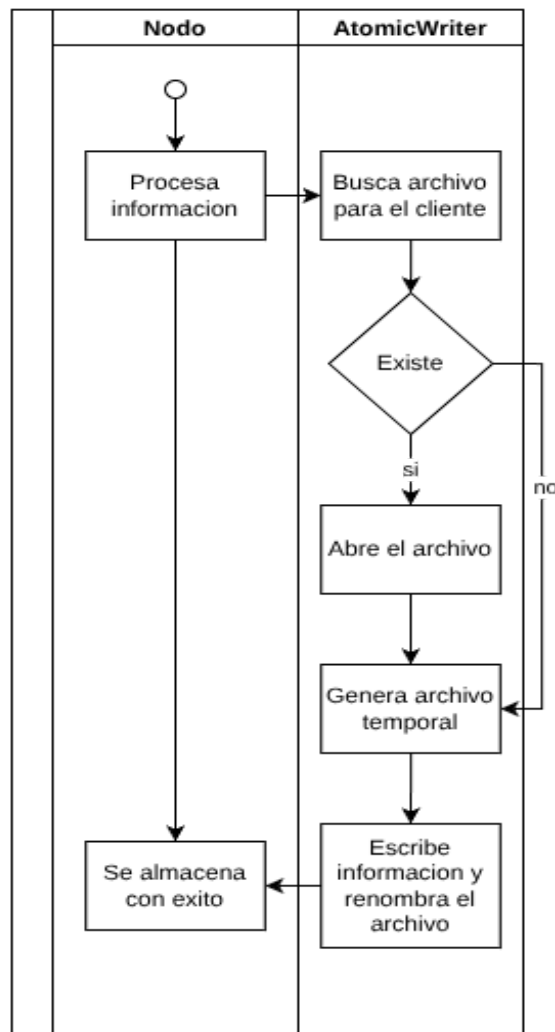
Nodos Group Top K

Representan un grupo particular de los nodos group, ya que los nodos en vez de procesar la información correspondiente y almacenarla, proceden a realizar el guardado de la información cruda del batch completo. Esto se debe al costo computacional que tiene procesar información, serializar y almacenarla, siempre usando un mismo archivo con el estado general de la transacción para un cliente.

Nodos Join

Los nodos join serializan tanto las tablas de soporte, como la tabla que contiene el resultado y también la caché del procesamiento de los archivos con el cual estuvo trabajando para llegar al resultado que contiene al momento de la bajada a memoria

El siguiente gráfico muestra un esquema básico del funcionamiento del esquema de escritura atómico general a todos los nodos



La información asociada a un nodo se mantiene grabada siempre y cuando este nodo sea cerrado de manera indebida, en caso de que un nodo sea cerrado de manera graceful la información persistida en el sistema de archivos del nodo será borrada al interpretar que él mismo quiso ser apagado de manera voluntaria.

Idempotencia

Una característica fundamental para el funcionamiento confiable de nuestro procesamiento de datos es que el sistema se comporte de manera adecuada ante situaciones inesperadas.

Una de estas situaciones inesperadas es el procesamiento duplicado de datos.

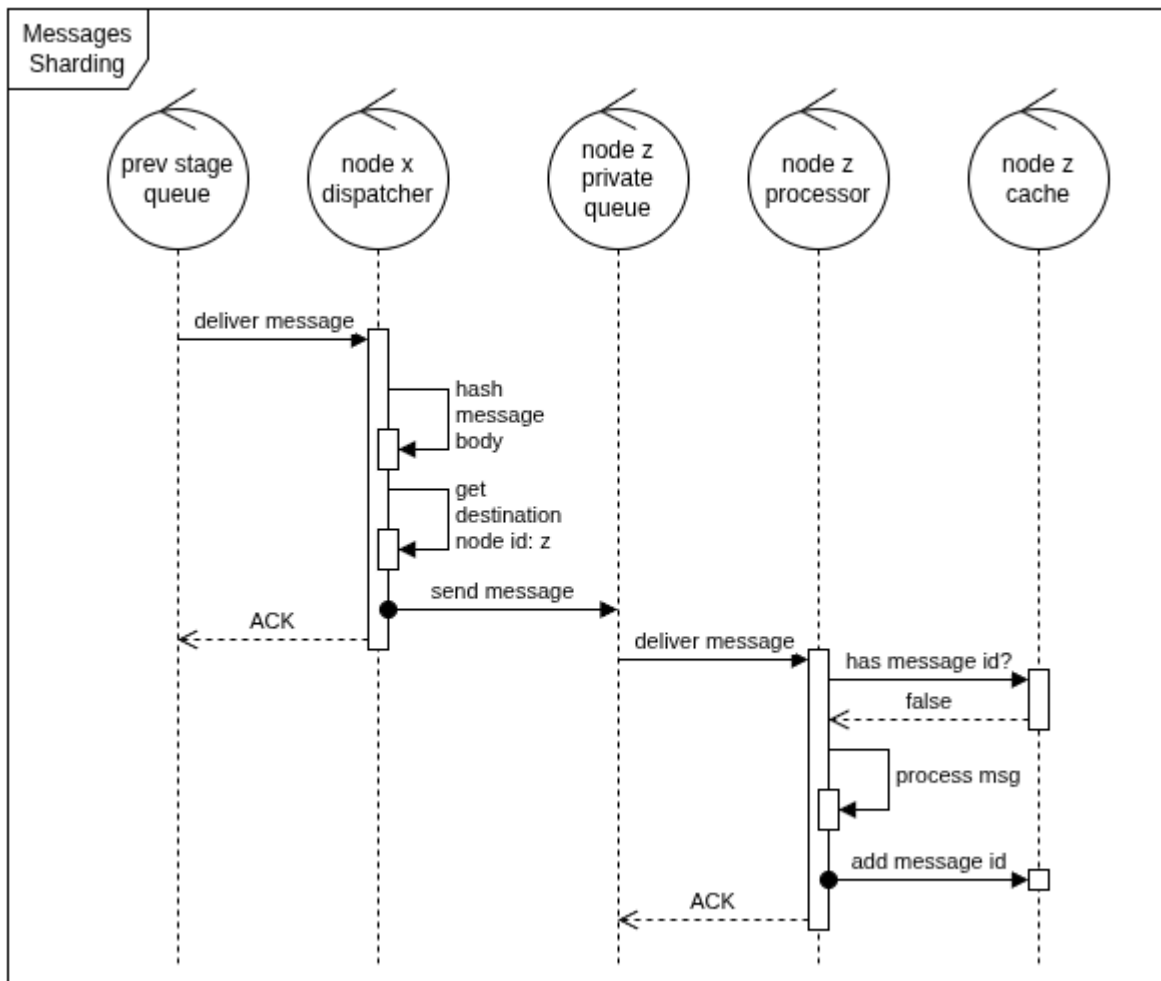
Para lograr idempotencia en el procesamiento de datos optamos por una combinación de estrategias que nos permiten entregar un mismo mensaje siempre al mismo receptor, sin importar cuantas veces suceda, de forma determinística y equilibrada.

Supongamos que tenemos 1000 mensajes y 5 nodos que van a filtrar estos mensajes, cada uno haciendo exactamente la misma tarea. Con nuestra estrategia logramos que cada nodo reciba aproximadamente 200 mensajes. Además, esos 200 mensajes serán siempre los mismos.

En el caso de que los mensajes estén repetidos dentro de una misma sesión de procesamiento, podemos identificar si ya fueron procesados y evitar reprocesarlos y continuar duplicando datos.

Como cada mensaje siempre termina en el mismo nodo, alcanza con verificar si el nodo receptor ya lo procesó, ya que nadie más lo pudo haber recibido.

A continuación presentaremos un diagrama que muestra el flujo de un mensaje cualquiera llegando a un nodo para ser procesado:



Prev stage queue:

- Es la cola que entrega los mensajes que fueron enviados por la etapa anterior.
- En este momento, los mensajes se entregan a cualquier nodo.

Node dispatcher:

- Aplica una función de hash SHA-256 sobre el body de mensaje
- Ese resultado se utiliza como identificador único del mensaje, llamemoslo `message_id`
- A partir del `message_id`, se calcula una operación módulo sobre la cantidad de nodos.
 - El resultado es el identificador del nodo que procesará el mensaje
- Con estos datos, ya podemos enviar a la cola privada del responsable del mensaje
 - En este ejemplo, es el *nodo z*

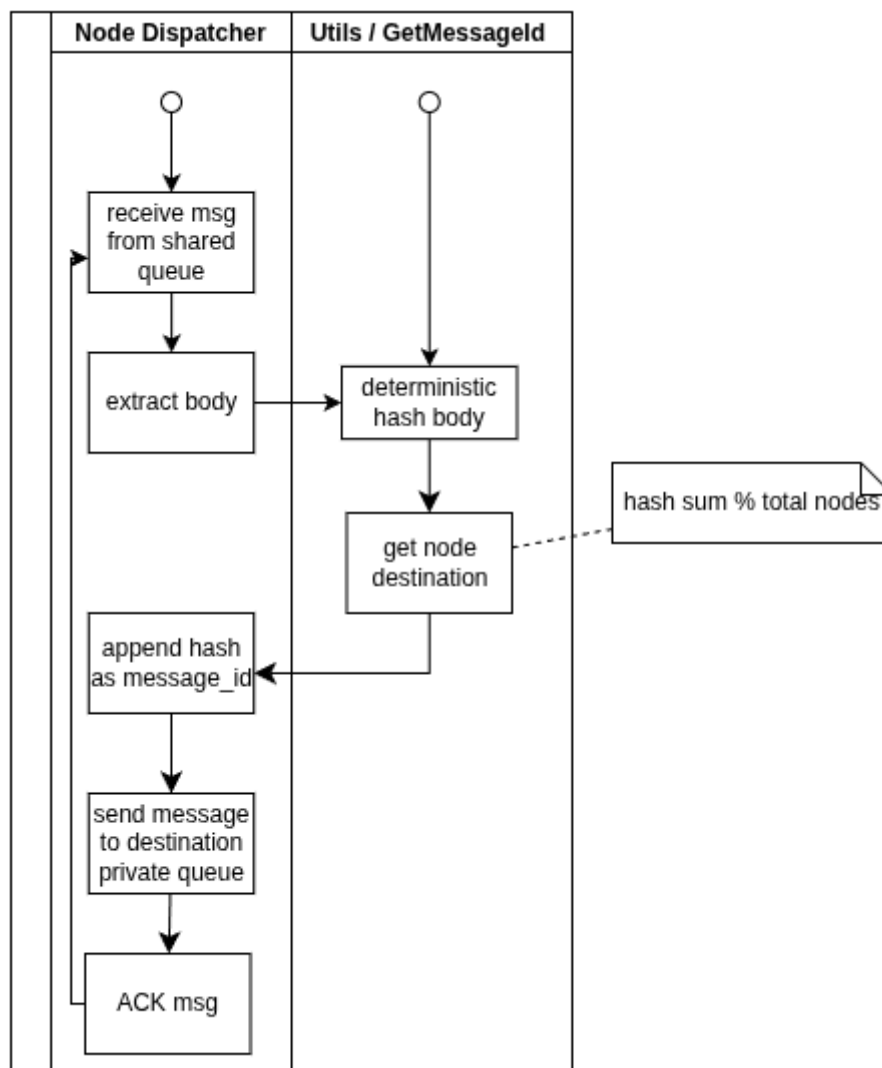
Node processor:

- Procesará el mensaje entregado a su cola privada
- Primero chequea si ya procesó ese `message_id` contra su caché
 - Si ya lo procesó no hace nada y envía el ACK
- Realiza el procesamiento del mensaje
- Guarda en la caché el `message_id` como procesado
- Termina enviando ACK

Para esta estrategia estamos confiando en el algoritmo SHA-256 y que no existirán colisiones. En caso de haberlas, los resultados no serán consistentes y deberíamos reemplazar el algoritmo.

Por último, puede darse el caso de que dos mensajes sean exactamente iguales y entonces el resultado del algoritmo de hash será el mismo, lo cual es esperable, pero consideramos que las probabilidades de que suceda son demasiado bajas como para tenerlo en cuenta.

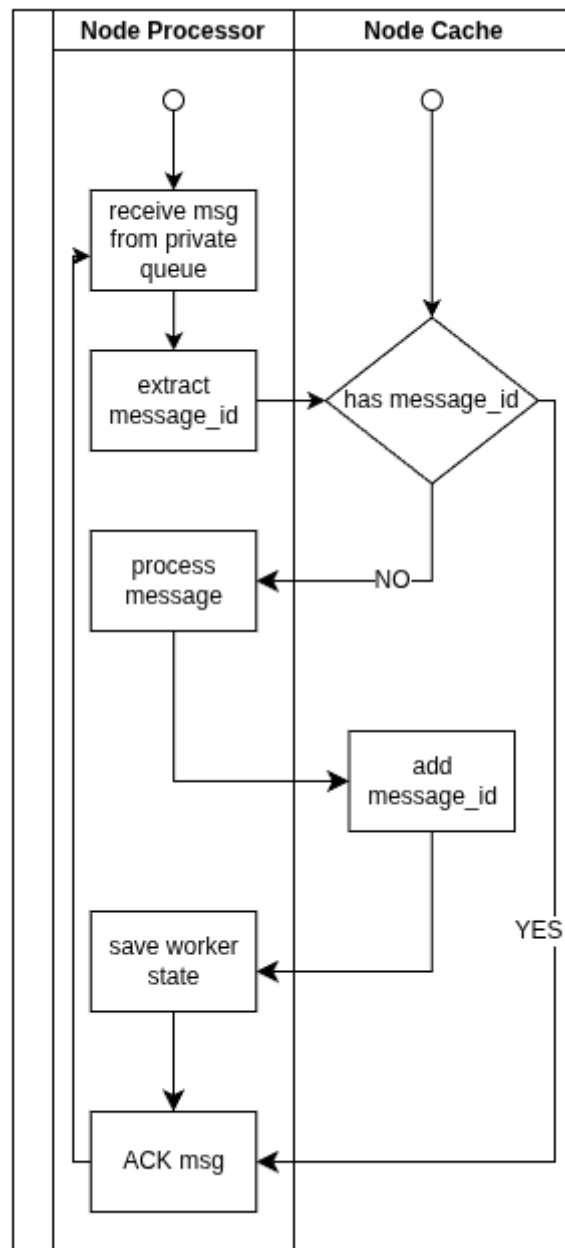
Agregamos unos diagramas de actividad:



Acá destacamos que los Node Dispatcher, presentes en cada nodo, son totalmente idénticos y stateless. El hash se realiza sobre el body en bytes del mensaje, por lo tanto es una operación veloz.

Como propiedad interesante, agregan al mensaje de RabbitMQ la propiedad *message_id* para ser utilizada luego en conjunto con la caché. Es importante que el algoritmo de hash sea determinístico.

Una vez que el mensaje fue entregado a una cola privada, ya es responsabilidad de la misma. Así, cada nodo termina siendo “dueño” de los mensajes que hayan sido enviados a su cola privada.



El trade-off de este mecanismo es que si un nodo se cae, los mensajes que le corresponden no serán consumidos por otros nodos. Pero tener este mecanismo nos permite evitar múltiples consensos entre los nodos antes de procesar cada mensaje.

Además, consideramos que los nodos no se caen muy seguido. Y en caso de hacerlo, serán resucitados a la brevedad gracias a Watch Mesh.

Cache del sistema

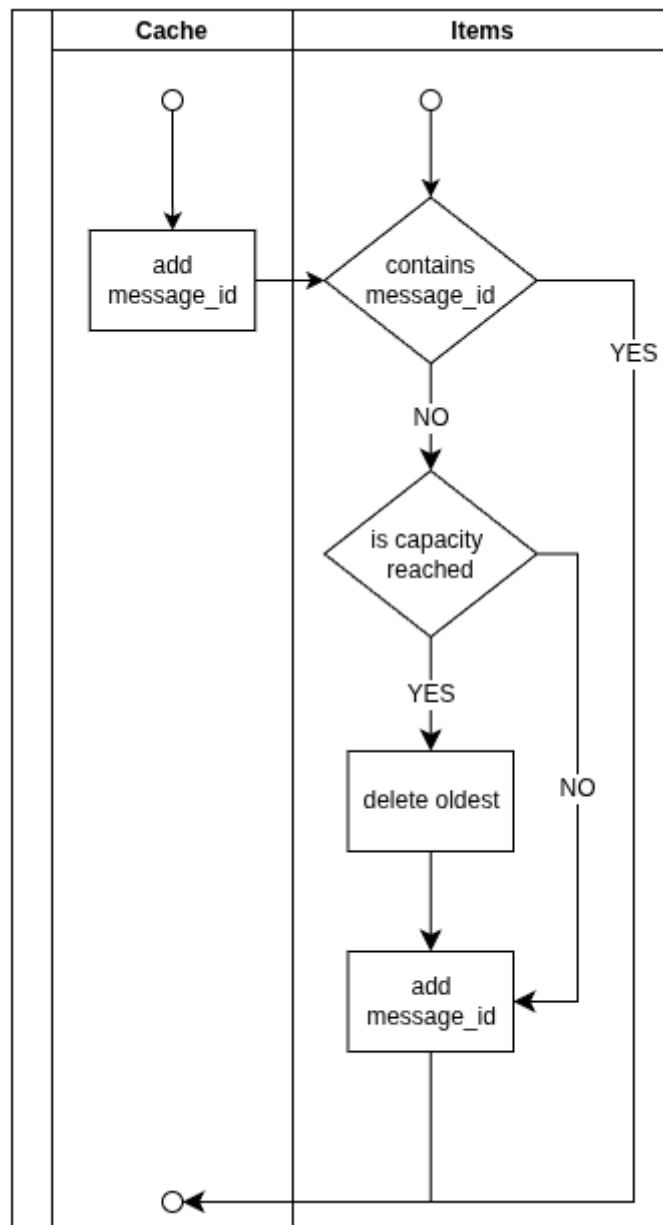
Para lograr idempotencia en todo el sistema fue necesario poder identificar los mensajes repetidos que recibe cada nodo, especialmente en aquellos que tienen efecto. Por ejemplo, un nodo de tipo filter puede procesar muchas veces el mismo mensaje pero no altera sus resultados. Un nodo de tipo group no tiene esa libertad.

Para ello creamos una estructura de tipo caché para almacenar los mensajes procesados por cada nodo y, dentro de él, por cada cliente. Entonces cada cliente tiene una porción de caché en cada nodo.

Cache
+ capacity: int
+ items: []string
+ contains(string): bool
+ add(string): void

Guardar absolutamente todos los mensajes no escala, por eso tiene una capacidad limitada, ahora configurada en 1000. Este número se eligió considerando la cantidad de batches (por lo tanto mensajes) que son generados por un solo archivo de transaction_items, que son los más grandes.

Estamos asumiendo que no habrán mensajes repetidos con diferencia de 1000. En caso de que eso suceda, consideramos que es un mal comportamiento del cliente. Del lado de los nodos junto a RabbitMQ y la técnica de requeues, los mensajes que pueden ser repetidos, lo hacen en ventanas mucho menores.



La decisión de limitar el tamaño de la caché también corresponde en que por cada mensaje procesado, se debe escribir en disco el estado actual.

Por último, se realizaron optimizaciones para que todas las operaciones sobre la caché sean de complejidad $O(1)$. Para el manejo del array utilizamos la técnica de buffer circular⁵ y una referencia en un diccionario a todos los elementos que contiene.

⁵ <https://docs.kernel.org/core-api/circular-buffers.html>

Crasher

Con el fin de testear la caída de los procesos en sectores críticos creamos una estructura que puede simular la terminación abrupta de un proceso.

Consta de una llamada a una función que determina si el proceso termina o no, a partir de un “dado” y un slot de tiempo según el id del nodo actual.

Esta función está presente en el código de cada nodo en múltiples lugares como los descritos en “Secciones de fallos”. Además, antes de cerrar el proceso deja un log con un mensaje que explica en dónde terminó.

Dado:

El crasher se inicializa con una probabilidad configurable, usualmente baja para evitar caídas demasiado recurrentes. Ante cada llamada al crasher, se genera un número de punto flotante aleatorio. Si el número generado es menor a la probabilidad asignada, tiene permitido avanzar. Caso contrario, se aborta la operación.

Slot de tiempo:

Para evitar que se caigan todos los nodos de una capa al mismo tiempo decidimos implementar un slot de tiempo en el que cada nodo se puede caer, con un tiempo de gracia en el medio para que Watch Mesh pueda revivirlos.

Se dividió el tiempo en 6 partes dentro de un minuto. Se toma el valor de segundo actual del minuto y el id del nodo. Se decide si el nodo termina abruptamente a partir de la siguiente tabla:

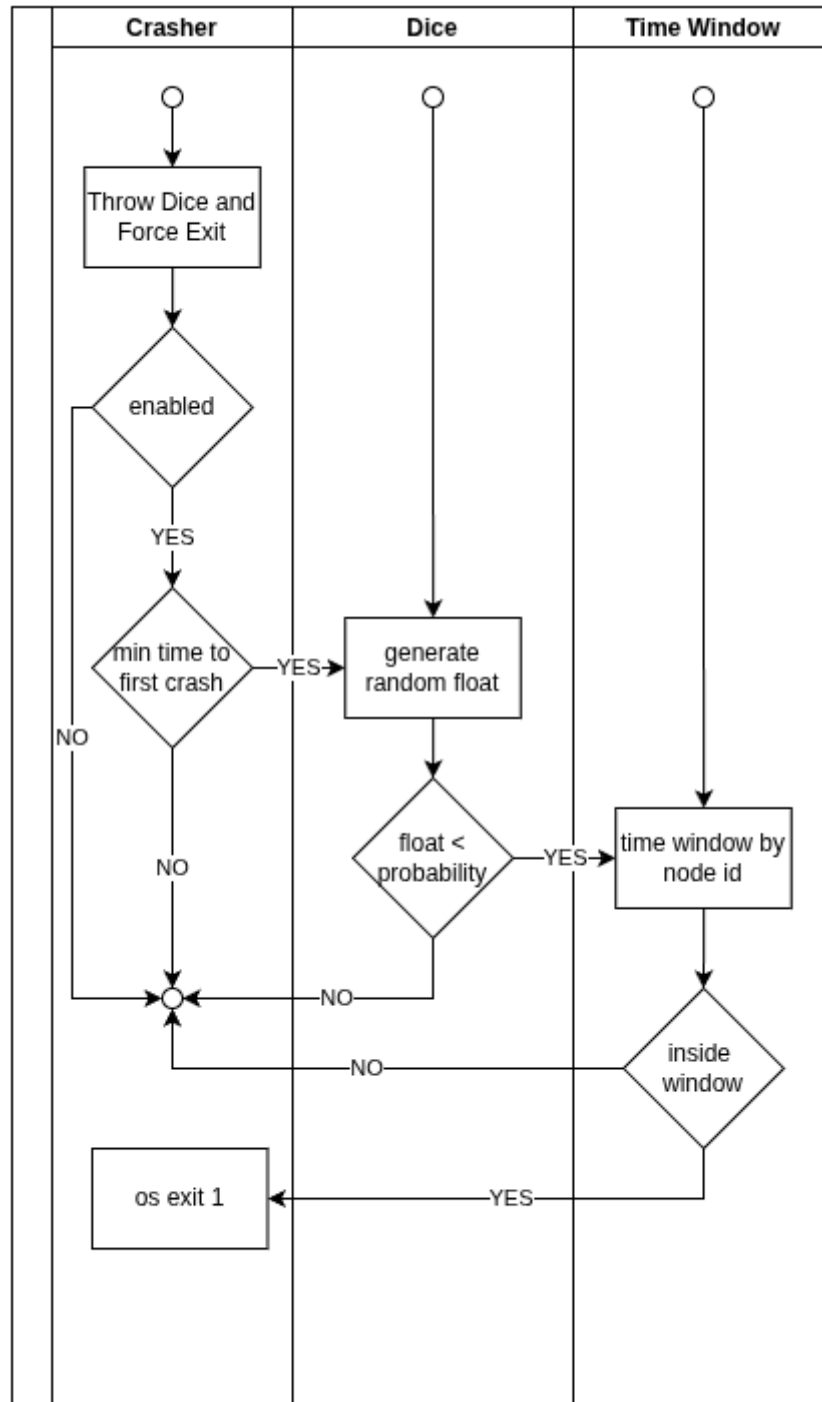
Time Window			
Start Time	End Time	Node ID	Exit
0 s	10 s	ID % 3 == 0	✓
10 s	20 s	ANY	X
20 s	30 s	ID % 3 == 1	✓
30 s	40 s	ANY	X
40 s	50 s	ID % 3 == 2	✓
50 s	60 s	ANY	X

Además, antes del dado tenemos dos instancias en donde se puede cancelar la caída:

- Si el crasher no está habilitado por configuración

- Si el nodo acaba de crearse le damos unos segundos de gracia para recuperarse antes de la próxima caída.

Entonces, el flujo completo quedaría de esta manera.



Simulacion de caos

Para simular caos en el sistema se utiliza un script que junto con la feature boom hacen el trabajo de atacar constantemente las capas del sistema tomando las precauciones necesarias, en escencia **chaos_monkey.sh** realiza un ataque ordenado sobre cada tipo de nodos de manera ordenada de principio a fin, utilizando un intervalo que el usuario provee desde la terminal y una cantidad de rondas que también se provee desde la terminal. Básicamente se ataca un nodo, por ejemplo, del tipo **filter-year** y una vez este nodo cierra de manera ungraceful (usando sig kill) se procede a esperar el intervalo configurable antes de proceder a atacar la siguiente capa.

El propósito de la herramienta es confirmar mediante un conjunto de pruebas variando la cantidad de rondas y los intervalos, el correcto funcionamiento del sistema ante un escenario de fallas inesperadas.

Es importante destacar que el script de caos tiene la capacidad de detectar si un nodo es el único nodo vivo de una capa y en ese escenario no se ataca el nodo escogido como víctima para la prueba que se está realizando, esperando así que la capa se recupere y pueda ser probado de manera correcta en la siguiente ronda de ataque en caso de haber un solo nodo

9. Referencias

- [1] *RabbitMQ Documentation* (s. f.). RabbitMQ. Recuperado en septiembre de 2025, de <https://www.rabbitmq.com/docs>
- [2] Chaos Monkey (s. f.). Netflix. Recuperado en octubre de 2025, de <https://netflix.github.io/chaosmonkey/>